

EC301 · MID-SEM

THE PAPERS

Four mid-sem papers — 2022, 2023, 2024 and 2025 — worked through, plus a **predicted paper for September 2026**. Note that the course code and the paper's whole character changed in 2025, and that matters more than anything else on this page.

WHAT ACTUALLY GETS ASKED

4 PAPERS READ END TO END

Read this section before you read anything else, because this subject broke its own pattern in 2025. The 2022–2024 papers were information-theory papers: entropy, capacity, coding, five or six long numericals. The 2025 paper, set under the new code EC301, was almost entirely **waveforms and block diagrams** — eight short questions, barely a calculation in sight, except the two coding ones.

TOPIC	22	23	24	25	OF 4
Shannon–Fano and / or Huffman coding, with efficiency					4
Entropy or information rate of a source					3
Channel capacity — Shannon–Hartley numerical					3
Binary symmetric channel — model or capacity					3
PCM — sampling rate, bits, SQNR, signalling rate					2
Line coding — draw the waveforms					2
Power spectral density of a line code					2
Linear block or cyclic code — G, H, syndrome decoding					1
ISI and the Nyquist criterion for zero ISI					1
ASK / FSK / PSK waveforms					1
BPSK modulator and demodulator block diagram					1

TOPIC	22	23	24	25	OF 4
Duobinary signalling and its decoding					1
Block diagram of a digital communication system					1
Delta modulation / adaptive step size					2

FILLED = IT APPEARED THAT YEAR · TINTED ROWS CAME UP IN EVERY PAPER WE HAVE

READ IT THIS WAY

One thing has been on every single paper: source coding. Shannon–Fano, Huffman, or both. In 2025 it was *both*, as two separate five-mark questions — a third of the entire paper. Learn the procedure until it is automatic, and always finish with all four numbers: average length L , entropy H , efficiency $\eta = H/L$, and redundancy $1 - \eta$. Then check the Kraft sum, because the 2025 paper asked for it explicitly.

Then read the 2025 paper carefully, because it changed shape. Eight questions instead of five or six. Twenty of its thirty marks were drawing and explaining: line codes, ASK/FSK/PSK, a BPSK block diagram, a duobinary waveform, the system block diagram. The heavy information-theory numericals of 2022–2024 — capacity from SNR, delta-modulator entropy, PCM signalling rates — did not appear at all.

What that means for 2026. The safe reading is that 2025 set the new pattern under the new code, so waveforms and block diagrams carry most of the paper. The unsafe part is assuming the numericals are gone for good: they are still on the syllabus, they were the whole paper for three years running, and they are cheap to revise. Our predicted paper therefore follows the 2025 shape but keeps two numericals in it.

Where the drawing marks actually go. Every waveform question names a specific bit stream — 1011001 in 2025 — and expects the *grid*: bit boundaries ruled first, bits written above them, a whole number of carrier cycles per bit. A tidy grid is most of the mark, and it is the easiest mark on the paper to drop.

PREDICTED PAPER — SEPTEMBER 2026

OUR READING, WITH FULL SOLUTIONS

Our prediction, not a leak. It follows the 2025 shape — eight questions, thirty marks, mostly drawing and explaining — while keeping the two source-coding questions that have appeared every year and two numericals from the older papers. Full solutions on every part.

Sardar Vallabhbhai National Institute of Technology, Surat

Mid Semester Examination, September 2026 — *predicted*

Branch: Electronics and Communication Engineering

Session 2026–27 · Semester 5

DIGITAL COMMUNICATIONS (EC 301)

MAXIMUM MARKS: 30

TIME: 1 HOUR 30 MINUTES

ALL QUESTIONS COMPULSORY

Note. All questions are compulsory. Marks are in the right-hand column. Where a waveform is asked for, rule the bit boundaries and label them — a description in words will not score.

- 1 What is Inter Symbol Interference (ISI)? Why does it occur? Discuss Nyquist's first criterion for achieving zero ISI. C01

1+1+2

SOLUTION

What it is. ISI is the smearing of one transmitted symbol into the time slots of its neighbours, so that the value sampled for one bit is contaminated by the tails of the bits around it.

Why it occurs. Every real channel is **bandwidth limited**. A sharp rectangular pulse contains frequencies out to infinity; passing it through a channel that removes the high frequencies spreads it out in time. The pulse that arrives is wider than the one that was sent, so its tail is still present when the next symbol is sampled.

THE PICTURE THAT EXPLAINS IT

Shout a word in a large empty hall. The echo of the first word is still bouncing around when you say the second, and the listener hears a muddle. ISI is exactly that, in electrical form — and it gets worse the faster you talk, which is why ISI is what ultimately limits the bit rate.

Nyquist's first criterion. The cure is not to send rectangles. Instead send a pulse shaped so that it is **zero at every neighbouring sampling instant**:

$$p(t) = 1 \quad \text{at } t = 0$$

$$p(t) = 0 \quad \text{at } t = n T_b \quad \text{for every } n \text{ other than } 0$$

The pulse may spread out in time as much as it likes — at the exact moments when its neighbours are sampled, it contributes nothing. In the frequency domain the same condition reads:

$$\sum \text{over } k \text{ of } P(f + k/T_b) = T_b \quad (\text{constant, for all } f)$$

which says the shifted copies of the pulse spectrum must add up to a flat line.

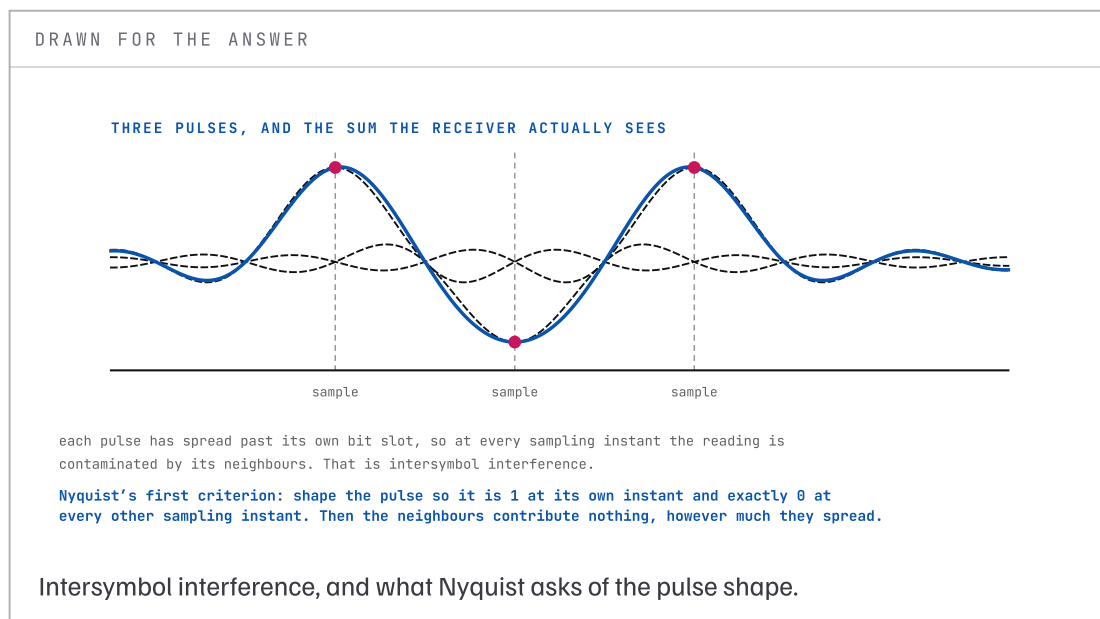
The minimum bandwidth this allows is the Nyquist bandwidth:

$$B_{\min} = R_b / 2 \quad \text{Hz}$$

The ideal brick-wall (sinc) pulse achieves exactly that, but it is unbuildable and its tails decay so slowly that a tiny timing error re-introduces huge ISI. The practical answer is the **raised cosine** pulse:

$$B = (R_b / 2)(1 + \alpha) \quad 0 \leq \alpha \leq 1$$

with the roll-off factor α trading bandwidth against tolerance of timing error. Real systems use $\alpha = 0.2$ to 0.5 .



- 2 Design the Unipolar NRZ, Unipolar RZ, Polar NRZ, Polar RZ, Bipolar (AMI) and Manchester line coding schemes for the bit sequence **1 0 1 1 0 0 1**. Which of these methods can be used for error detection, and which can be used for time synchronisation? C01

3+2

SOLUTION

Rule the seven bit boundaries first, and for the RZ codes mark the half-bit points too.

bit	:	1	0	1	1	0	0	1
UNIPOLAR NRZ (1 = +A whole bit, 0 = 0)								
+A		---		---		---		---
0		-----		-----		-----		-----
UNIPOLAR RZ (1 = +A for half a bit, 0 = 0)								
+A		_		_		_		_
0			-----			-----		
POLAR NRZ (1 = +A, 0 = -A)								
+A		---		---		---		---
0								
-A		-----		-----		-----		-----
POLAR RZ (1 = +A/2 half bit, 0 = -A/2 half bit, then 0)								
+A/2		_		_		_		_
0			---			---		
-A/2								
BIPOLAR AMI (1s alternate +A / -A, 0 = 0)								
+A		---		---		---		---
0			-----			-----		
-A		-----		-----		-----		-----
		(+A)	0	(-A)	(+A)	0	0	(-A)
MANCHESTER (1 = low-to-high mid-bit, 0 = high-to-low mid-bit)								
+A		_		_		_		_
0								
-A								

For AMI, pencil the polarities before you draw. The 1s are at positions 1, 3, 4 and 7, so they take +A, -A, +A, -A in that order. The 0s sit on the zero line and are skipped in the counting.

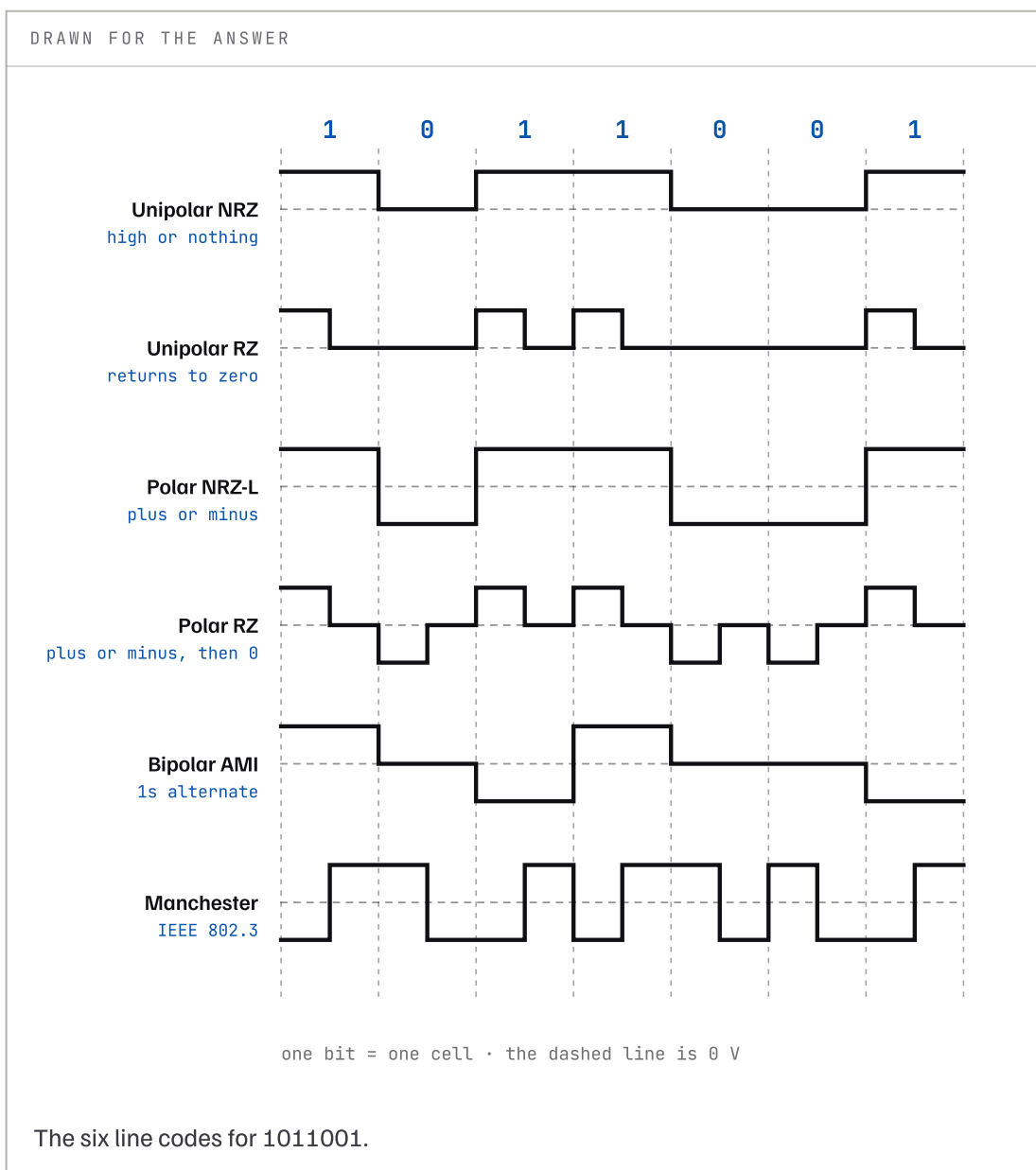
Which can detect errors? **Bipolar AMI**

AMI has a built-in rule: successive 1s must alternate in polarity. If the receiver ever sees two +A pulses in a row with no -A between them, that is an **AMI violation** and an error has occurred. No extra bits are needed — the detection is free, and it comes from the coding itself.

Which can synchronise the clock?

Manchester (and, partly, the RZ codes)

- **Manchester is fully self-synchronising:** there is a guaranteed transition in the middle of *every* bit, whatever the data, so the receiver can always recover the clock. That is why it costs double the bandwidth.
- **RZ codes** give a transition inside every 1, which helps, but unipolar RZ gives nothing during a long run of 0s.
- **NRZ codes are the worst:** a long run of identical bits holds the line flat, the receiver's clock drifts, and it loses count. This is exactly why scramblers exist.

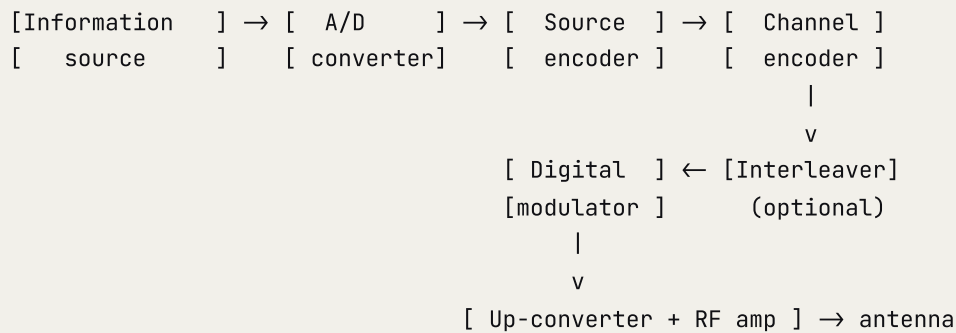


3 Design the block diagram of a digital communication system and explain the work of each block. C01

3

SOLUTION

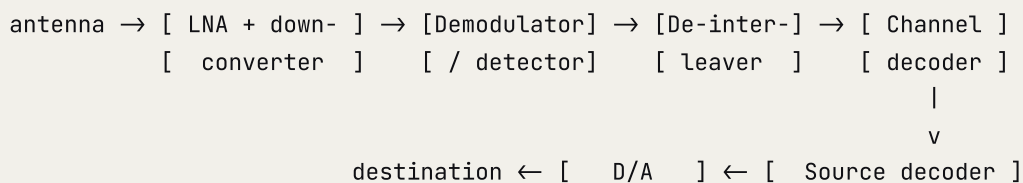
TRANSMITTER



CHANNEL $y(t) = x(t) + n(t)$

impairments: path loss, multipath fading, AWGN, interference, distortion

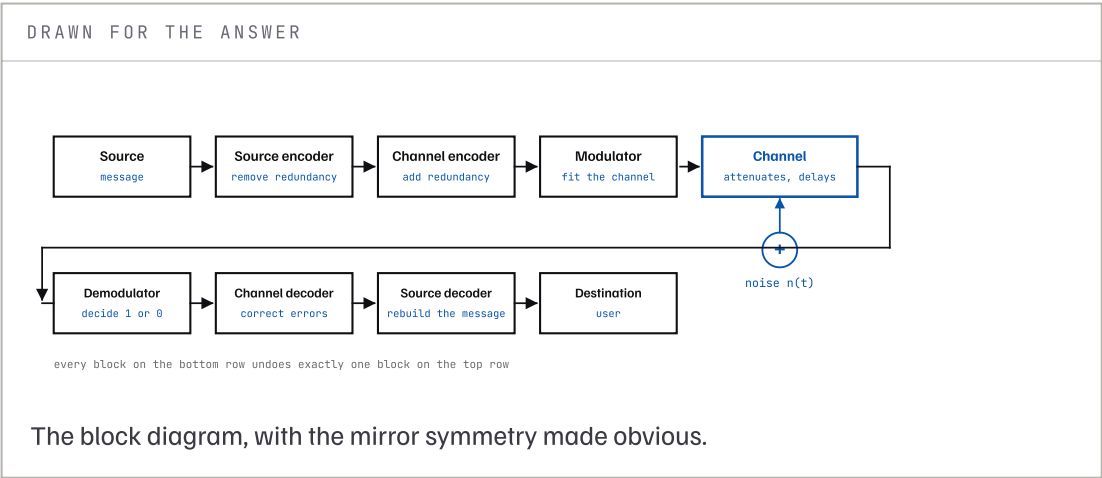
RECEIVER



BLOCK	ITS JOB
Information source	Produces the message — microphone, camera, keyboard, sensor.
A/D converter	Sampling, quantization and encoding — turns a smooth wave into bits.
Source encoder	Squeezes the data. Removes the redundancy already in it, so fewer bits are needed. ZIP, JPEG, MP3.
Channel encoder	Adds carefully designed redundancy so the receiver can detect and repair errors.
Interleaver	Shuffles the bit order, so that a burst of channel damage is spread thinly over many codewords — and thin damage is what error correction handles well.
Digital modulator	Maps groups of bits onto waveforms (symbols): BPSK, QPSK, 16-QAM.
Up-converter, RF amp	Shifts to the radio frequency the antenna can radiate, and supplies the power to survive the journey.
Channel	Air, cable, fibre or satellite link — where all the damage happens.

BLOCK	ITS JOB
LNA, down-converter	The arriving signal is tiny, so it is amplified by a <i>low-noise</i> amplifier first, then shifted back down.
Demodulator / detector	Decides which symbol was sent. This decision is a guess, and wrong guesses are where bit errors are born.
Channel decoder	Uses the added redundancy to find and fix the wrong guesses.
Source decoder	Un-squeezes back to full size.
D/A converter	Turns the numbers back into a wave for the speaker or screen.

The mirror symmetry is the point. Every receiver block exactly undoes the matching transmitter block, in the reverse order. Say that, and add the apparent contradiction and its answer: source coding *removes* redundancy and channel coding *adds* it, because the first removes useless accidental redundancy and the second adds a small amount of mathematically designed redundancy built to fight noise.



4

Design the waveforms of ASK, FSK and PSK for the bit sequence **1 0 1 1 0 0**

2

1.

C01

SOLUTION

bit : 1011001

ASK (on-off keying)
 /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\

 on off on on off off on

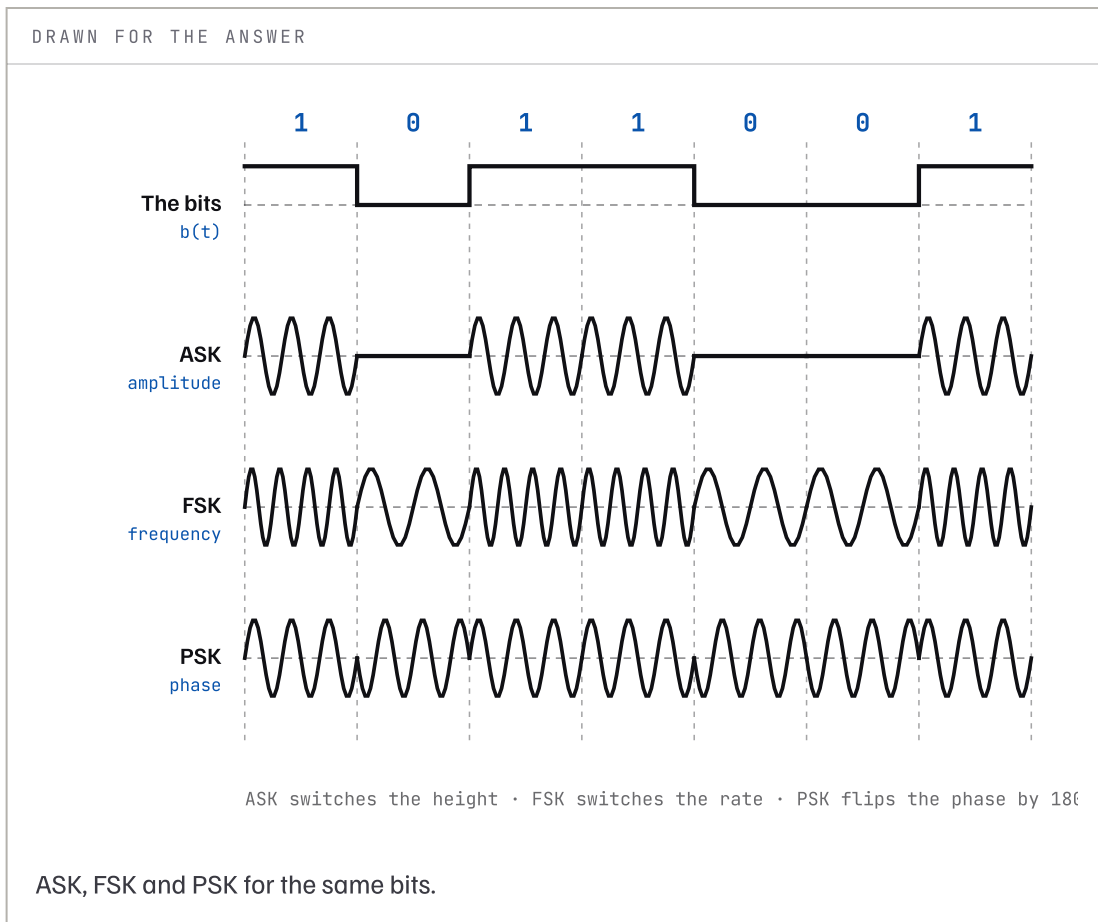
FSK (two tones, same amplitude throughout)
 /\ /\ /\ /--\ /\ /\ /\ /\ /\ /--\ /--\ /\ /\ /\
 fast slow fast fast slow slow fast
 f1 f2 f1 f1 f2 f2 f1

PSK (same amplitude, same frequency, phase flips by 180 deg)
 /\ /\ /\ \/\ /\ /\ /\ /\ \/\ /\ /\ \/\ /\ /\ \/\ /\ /\
 0 deg 180deg 0 deg 0 deg 180deg 180deg 0 deg

	WHAT VARIES	WHAT STAYS FIXED	FOR A 0
ASK	amplitude	frequency, phase	carrier switched off — a flat line
FSK	frequency	amplitude, phase	the second tone f ₂ , drawn with fewer cycles
PSK	phase	amplitude, frequency	the same wave inverted — it starts downwards

HOW TO DRAW THESE AND KEEP THE MARK

Rule the seven bit boundaries first. Put a **whole number of carrier cycles in each bit interval** — two or three is plenty. For PSK, draw the carrier for a 1 and the same wave upside down for a 0, so the waveform visibly jumps at every change of bit. Neatness on the grid is most of the mark here.



5 Design the block diagram of BPSK modulation and demodulation. Explain the work of each block. C01

3

SOLUTION

MODULATOR

```

binary data ---> [ NRZ level ] ---> (X) ---> [ band pass ] ---> BPSK out
(unipolar)      [ converter ]      ^      [ filter ]
                                |
                                [ carrier oscillator ]
                                (balanced modulator)
  
```

DEMODULATOR (coherent)

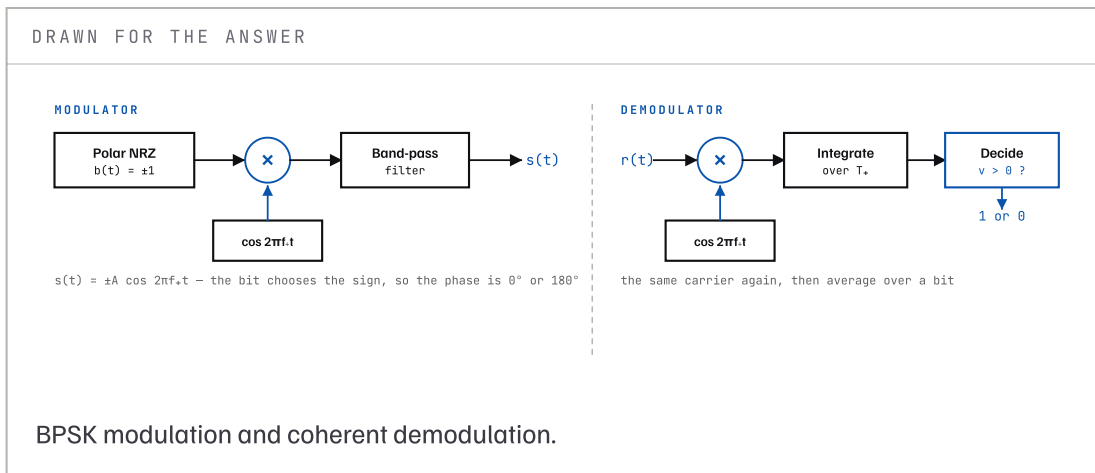
```

BPSK in ---> [ band pass ] ---> (X) ---> [ low pass ] ---> [ sample ] ---> data
[ filter ]      ^      [ filter ]      [ + decide ]
                |
                [ carrier recovery ]
  
```

BLOCK	ITS JOB
NRZ level converter	Turns unipolar data (0 V, +V) into polar data (−V, +V). This is the step that makes the phase flip possible — a 0 must become a <i>negative</i> number, not zero.
Carrier oscillator	Generates the reference sine wave at f_c .
Balanced modulator	A product modulator. Multiplying by +V leaves the carrier alone; multiplying by −V inverts it — which <i>is</i> the 180° phase shift. Called <i>balanced</i> because the carrier itself is suppressed in the output.
Band pass filter (Tx)	Removes the out-of-band products and limits the transmitted spectrum to its channel.
Band pass filter (Rx)	Passes the wanted channel, rejects noise and neighbouring channels before anything else is done.
Carrier recovery	Regenerates a local carrier locked in frequency and phase to the incoming one. BPSK is coherent, so this is essential — a 180° lock error inverts every bit.
Balanced modulator (Rx)	Multiplies the received signal by the recovered carrier. Since $\cos^2\theta = \frac{1}{2}(1 + \cos 2\theta)$, this gives the wanted baseband term plus a component at $2f_c$.
Low pass filter	Throws away the $2f_c$ component and leaves the data.
Sample and decide	Samples once per bit at the best instant and compares with a threshold of zero: positive → 1, negative → 0.

THE EXTRA SENTENCE WORTH WRITING

Because +cos and −cos look identical to a recovery loop that has just started, the receiver can lock 180° out and decode every bit backwards. The standard cure is **differential encoding** — send the *change* rather than the value — which gives DPSK, at a cost of roughly 3 dB in noise performance.



- 6 Draw the duobinary signal for the sequence **1 0 1 1 0 1**. Explain how the same sequence is decoded at the receiver from the received duobinary signal. C 01

2+1

SOLUTION

Step 1 — precode. Duobinary is used with a precoder so that a single sample error cannot propagate. The precoder is a running XOR:

$$p(k) = d(k) \text{ XOR } p(k-1) \quad \text{starting from } p(-1) = 0$$

d :	1	0	1	1	0	1
p :	1	1	0	1	1	0

$p_0 = 1 \text{ xor } 0 = 1$
 $p_1 = 0 \text{ xor } 1 = 1$
 $p_2 = 1 \text{ xor } 1 = 0$
 $p_3 = 1 \text{ xor } 0 = 1$
 $p_4 = 0 \text{ xor } 1 = 1$
 $p_5 = 1 \text{ xor } 1 = 0$

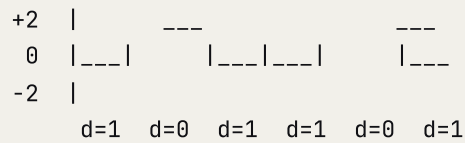
Step 2 — map to polar and add the neighbour. Take $1 \rightarrow +1$ and $0 \rightarrow -1$, then form $y(k) = a(k) + a(k-1)$, which is the controlled ISI duobinary deliberately introduces:

a :	+1	+1	-1	+1	+1	-1	$(a(-1) = -1, \text{ since } p(-1) = 0)$
-----	----	----	----	----	----	----	--

$y(0) = +1 + (-1) = 0$
 $y(1) = +1 + (+1) = +2$
 $y(2) = -1 + (+1) = 0$
 $y(3) = +1 + (-1) = 0$
 $y(4) = +1 + (+1) = +2$
 $y(5) = -1 + (+1) = 0$

y :	0	+2	0	0	+2	0
-----	---	----	---	---	----	---

The waveform takes only three levels — -2 , 0 and $+2$:



Step 3 — decoding, which is the last mark. The decision rule needs no memory at all:

$$\begin{aligned} \text{if } y(k) = 0 & \rightarrow d(k) = 1 \\ \text{if } y(k) = +2 \text{ or } -2 & \rightarrow d(k) = 0 \end{aligned}$$

```
y :    0    +2    0    0    +2    0
d :    1     0    1    1     0    1    -- the original sequence
```

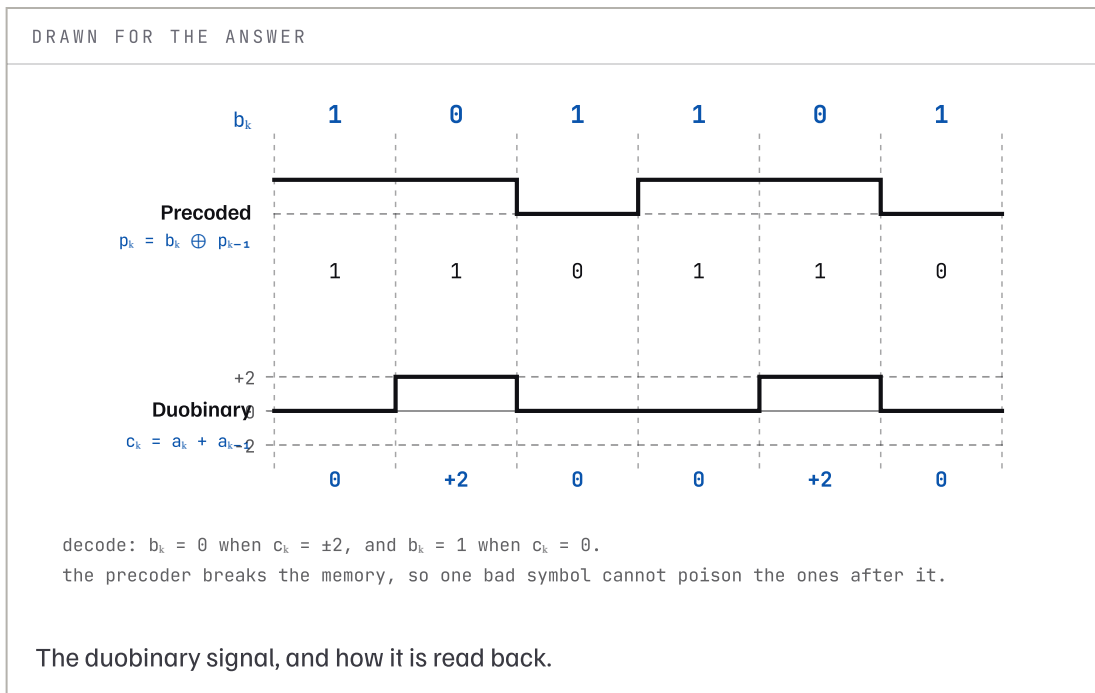
Recovered exactly: 1 0 1 1 0 1

TWO THINGS WORTH ADDING

Why precode at all. Without the precoder, decoding needs the *previous* decision, so one error would propagate forever. With it, every symbol is decided on its own sample alone.

Free error detection. Successive non-zero samples must alternate in polarity when separated only by zeros. Two +2s in a row with only zeros between them means an error has occurred — no extra bits required.

And the real prize: duobinary achieves the theoretical minimum bandwidth $R_b/2$ with a *realisable* filter, which the ideal brick wall is not.



- 7 The probabilities of occurrence of five symbols are $P(A) = 0.35$, $P(B) = 0.20$, $P(C) = 0.15$, $P(D) = 0.15$, $P(E) = 0.10$, $P(F) = 0.05$. Construct the codes according to **Shannon–Fano** coding. Calculate the average code length, the entropy of the source, the efficiency and the redundancy. Further, check whether Kraft's inequality is satisfied. C04

5

SOLUTION

Step 1 — sort by probability (already sorted) and split into two groups of as nearly equal probability as possible.

symbol	P	cumulative
A	0.35	0.35
B	0.20	0.55
C	0.15	0.70
D	0.15	0.85
E	0.10	0.95
F	0.05	1.00

split 1: {A,B} = 0.55 vs {C,D,E,F} = 0.45 (gap 0.10, the closest)
 top group → bit 0, bottom group → bit 1

split 2a: {A} = 0.35 vs {B} = 0.20 → A = 00, B = 01
 split 2b: {C,D} = 0.30 vs {E,F} = 0.15 → C,D = 10, E,F = 11
 split 3a: {C} vs {D} → C = 100, D = 101
 split 3b: {E} = 0.10 vs {F} = 0.05 → E = 110, F = 111

Step 2 — the code.

SYMBOL	P	CODE	L	P × L
A	0.35	00	2	0.70
B	0.20	01	2	0.40
C	0.15	100	3	0.45
D	0.15	101	3	0.45
E	0.10	110	3	0.30
F	0.05	111	3	0.15

Step 3 — average code length.

$$L = 0.70 + 0.40 + 0.45 + 0.45 + 0.30 + 0.15 = 2.45 \text{ bits/symbol}$$

Step 4 — entropy.

$$\begin{aligned}
 H(X) &= - \sum P \log_2 P \\
 &= 0.35(1.5146) + 0.20(2.3219) + 0.15(2.7370) \\
 &\quad + 0.15(2.7370) + 0.10(3.3219) + 0.05(4.3219) \\
 &= 0.5301 + 0.4644 + 0.4106 + 0.4106 + 0.3322 + 0.2161 \\
 &= 2.3640 \text{ bits/symbol}
 \end{aligned}$$

Step 5 — efficiency and redundancy.

$$\begin{aligned}
 \eta &= H / L = 2.3640 / 2.45 = 0.9649 \rightarrow 96.49 \% \\
 \gamma &= 1 - \eta \rightarrow 3.51 \%
 \end{aligned}$$

Step 6 — Kraft's inequality.

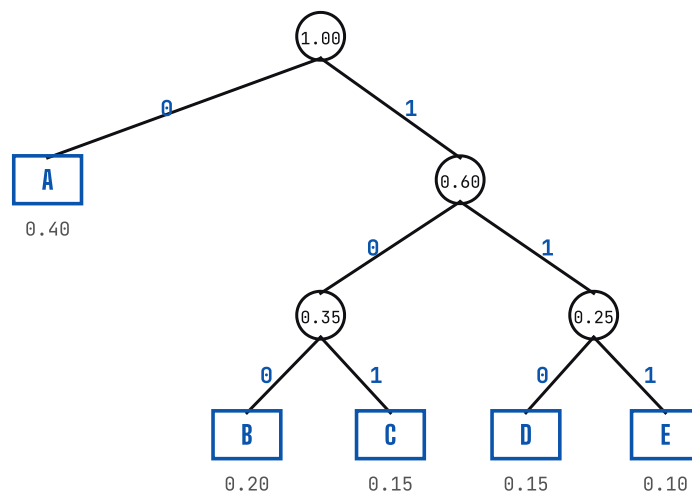
$$\begin{aligned}
 K &= \sum 2^{-L} \\
 &= 2^{-2} + 2^{-2} + 2^{-3} + 2^{-3} + 2^{-3} + 2^{-3} \\
 &= 0.25 + 0.25 + 0.125 + 0.125 + 0.125 + 0.125 \\
 &= 1.00 \leq 1 \quad \text{SATISFIED}
 \end{aligned}$$

$$L = 2.45 \cdot H = 2.364 \cdot \eta = 96.49\% \cdot \text{redundancy } 3.51\% \cdot \text{Kraft} = 1.00 \checkmark$$

Equality in Kraft means the code is **complete** — every leaf position in the binary tree is used, with no wasted branch. That is the best a prefix-free code can do.

DRAWN FOR THE ANSWER

SHANNON-FANO - SPLIT FROM THE TOP



The Shannon-Fano tree for this source.

- 8 For the same source, use **Huffman** coding instead. Calculate the average code length, entropy, efficiency and redundancy, and check Kraft's inequality. Compare the two codes. C04

5

SOLUTION

Step 1 — build the tree bottom up. At each stage merge the **two smallest** probabilities and re-sort.

```

start   : 0.35 0.20 0.15 0.15 0.10 0.05
merge 1 : 0.10 + 0.05 = 0.15      → 0.35 0.20 0.15 0.15 0.15
merge 2 : 0.15 + 0.15 = 0.30      → 0.35 0.30 0.20 0.15
merge 3 : 0.20 + 0.15 = 0.35      → 0.35 0.35 0.30
merge 4 : 0.35 + 0.30 = 0.65      → 0.65 0.35
merge 5 : 0.65 + 0.35 = 1.00      → done
  
```

Step 2 — assign bits by tracing back from the root (0 to the upper branch, 1 to the lower, by convention).

SYMBOL	P	CODE	L	P × L
A	0.35	0	1	0.35
B	0.20	111	3	0.60
C	0.15	100	3	0.45

SYMBOL	P	CODE	L	P × L
D	0.15	101	3	0.45
E	0.10	1100	4	0.40
F	0.05	1101	4	0.20

Step 3 — the four numbers.

$L = 0.35 + 0.60 + 0.45 + 0.45 + 0.40 + 0.20 = 2.45$ bits/symbol
 $H(X) = 2.3640$ bits/symbol (the source is unchanged)
 $\eta = 2.3640 / 2.45 = 96.49\%$
 $\gamma = 3.51\%$
 $Kraft = 2^{-1} + 3(2^{-3}) + 2(2^{-4})$
 $= 0.5 + 0.375 + 0.125 = 1.00 \leq 1$ SATISFIED

Step 4 — the comparison, which is what the question is really after.

	SHANNON–FANO	HUFFMAN
Direction	Top-down: repeatedly split	Bottom-up: repeatedly merge
Code lengths	2, 2, 3, 3, 3, 3	1, 3, 3, 3, 4, 4
Average length L	2.45	2.45
Efficiency	96.49%	96.49%
Optimal?	Not guaranteed	Always optimal

They tie here — different codewords, identical average length. That happens often with well-behaved probabilities, and it does not mean the two methods are equivalent.

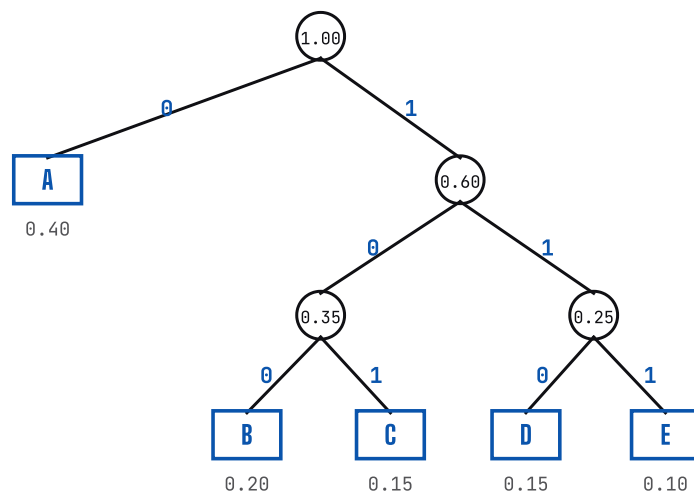
THE SENTENCE THAT EARNS THE COMPARISON MARK

Huffman is provably optimal; Shannon–Fano is not. Huffman can never be beaten for a given set of probabilities, whereas Shannon–Fano can fall short when the splits cannot be made evenly. Here they happen to tie, but on a less friendly source Shannon–Fano loses — which is exactly why JPEG, MP3 and ZIP all use Huffman.

Also worth saying: **Huffman codes are not unique.** Swapping the 0/1 labels or breaking a tie the other way gives different codewords with the same L. If your codewords differ from a friend's but your L matches, you are both right.

DRAWN FOR THE ANSWER

HUFFMAN - MERGE FROM THE BOTTOM



The Huffman tree for the same source.

MID-SEM, SEPTEMBER 2025

EC301 · 30 MARKS · THE PATTERN-BREAKING PAPER

Eight questions, and a completely different character from the three papers before it. Twenty of the thirty marks are drawing and explaining; the only numericals are the two source-coding questions.

Sardar Vallabhbhai National Institute of Technology, Surat

Mid Semester Examination September 2025

Branch: Electronics and Communication Engineering

Session: 2025-26 · Semester 5

DIGITAL COMMUNICATIONS (EC 301)

MAXIMUM MARKS: 30

TIME: 1 HOUR 30 MINUTES

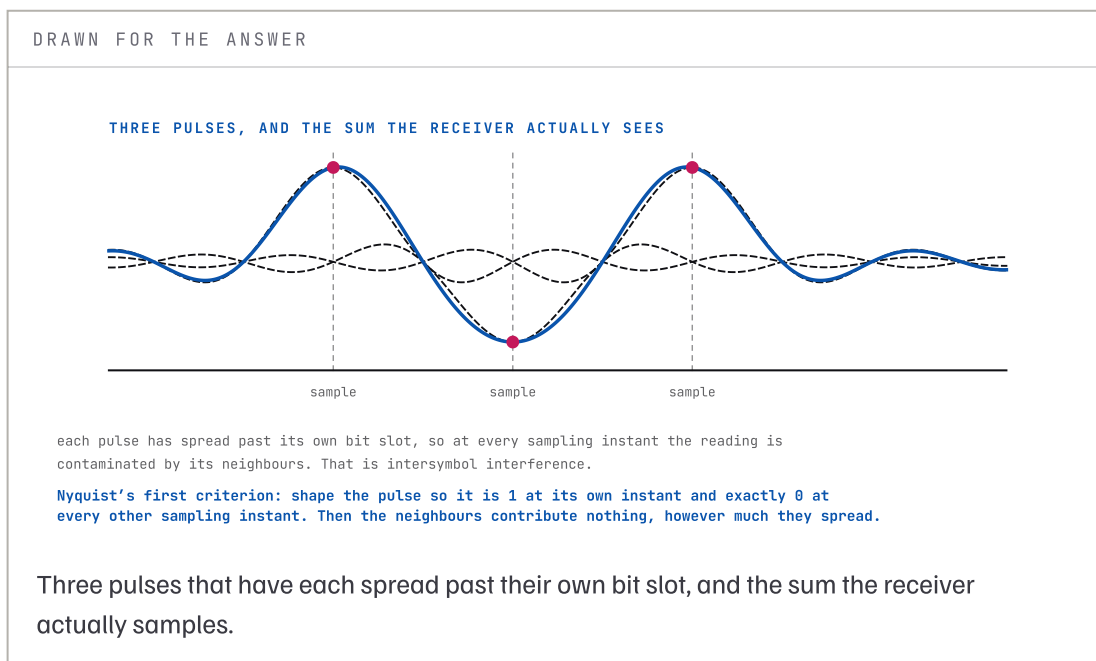
ALL QUESTIONS COMPULSORY

- 1 What is Inter Symbol interference (ISI)? Why it occurs? Discuss Nyquist's first criteria to achieve Zero ISI. C01

1+1+2

SOLUTION

Worked in full at the predicted Q1 above: what ISI is, the bandwidth-limiting cause, Nyquist's zero-crossing condition in both time and frequency, the $R_b/2$ minimum bandwidth, and the raised cosine as the practical answer.



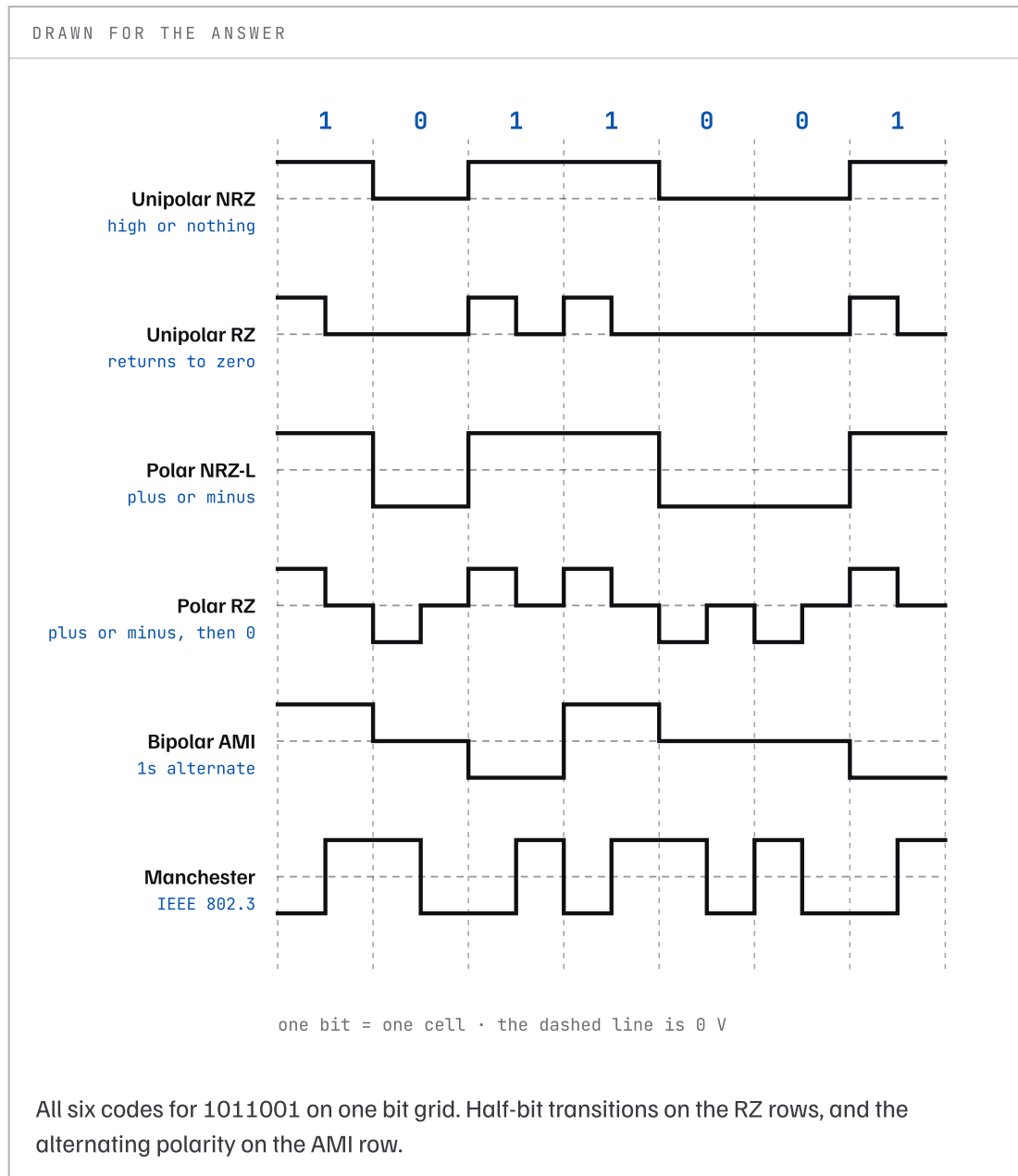
- 2 Design the Unipolar NRZ, Unipolar RZ, Polar NRZ, Polar RZ, Bipolar and Manchester line coding schemes for the following sequence of bits: **1 0 1 1 0 0**

3+2

1. Which of the above methods can be used for error detection and which methods can be used for time synchronization? C01

SOLUTION

Worked in full at the predicted Q2 above, with all six waveforms on the same bit grid. The two answers the second part wants: **bipolar AMI** for error detection (an AMI violation is a free error flag) and **Manchester** for synchronisation (a guaranteed transition in every bit, whatever the data).

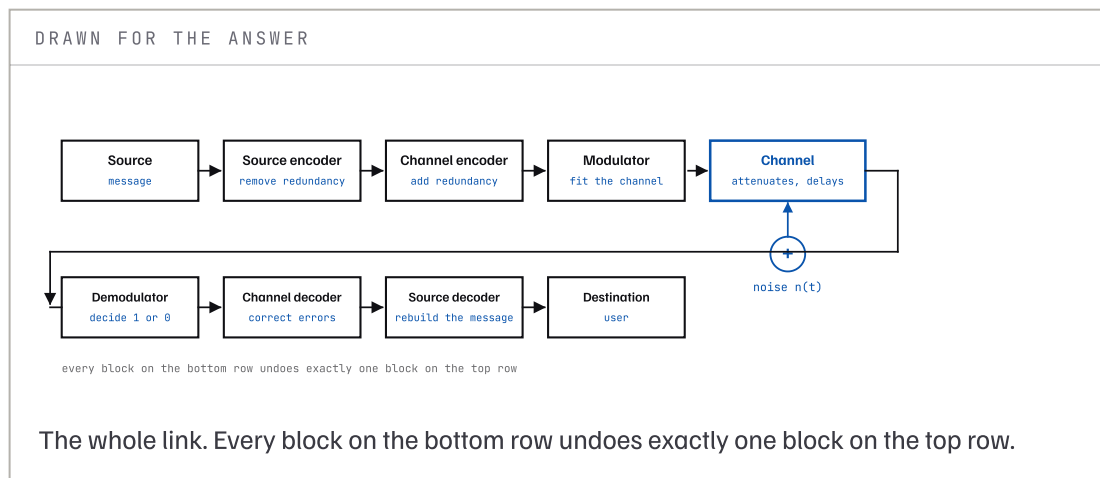


3 Design the block diagram of a digital communication system. Explain the work of each block. C01

3

SOLUTION

Worked in full at the predicted Q3 above — the full transmitter, channel and receiver chain with a one-line job description for each of the thirteen blocks, plus the mirror-symmetry observation and the source-versus-channel-coding contradiction resolved.

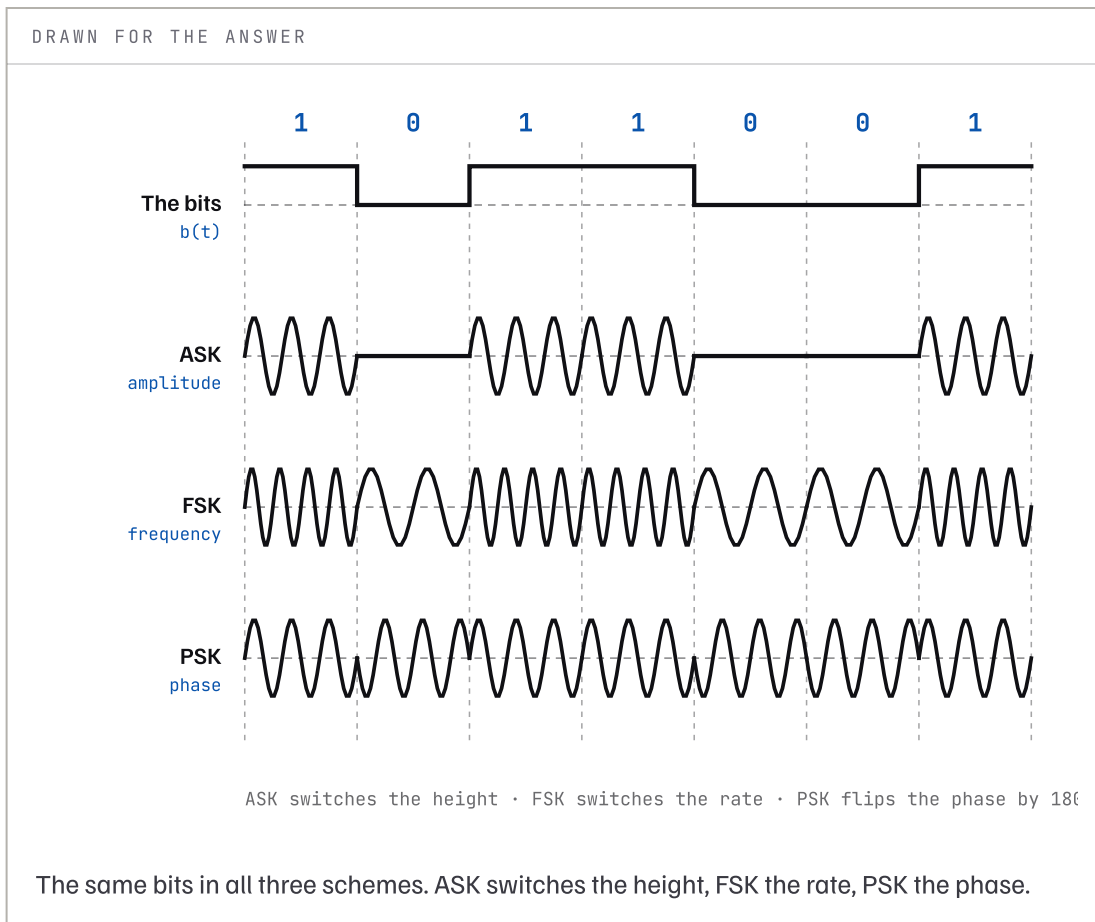


4 Design the waveforms of ASK, FSK and PSK for the following sequence of bits: **1 0 1 1 0 0 1**. C01

2

SOLUTION

Worked at the predicted Q4 above. Two marks for three waveforms means speed matters: rule the grid, two or three carrier cycles per bit, flat line for a 0 in ASK, fewer cycles for a 0 in FSK, inverted wave for a 0 in PSK.



5 Design the block diagram of BPSK modulation and Demodulation. Explain the work of each block. C01

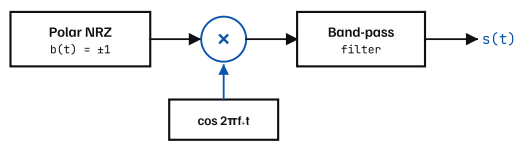
3

SOLUTION

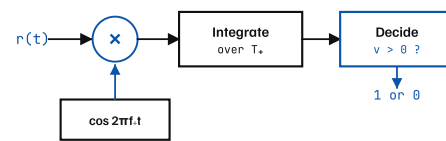
Worked in full at the predicted Q5 above, with both block diagrams and the nine-block table. Do not forget the **carrier recovery** block in the receiver — it is what makes BPSK coherent, and leaving it out is the usual way this question loses marks.

DRAWN FOR THE ANSWER

MODULATOR


 $s(t) = \pm A \cos 2\pi f_c t$ – the bit chooses the sign, so the phase is 0° or 180°

DEMODULATOR



the same carrier again, then average over a bit

The modulator and the coherent demodulator. The same carrier appears in both, which is what makes it coherent.

6 Draw a figure of a duobinary signal for the following sequence: **1 0 1 1 0 1**.

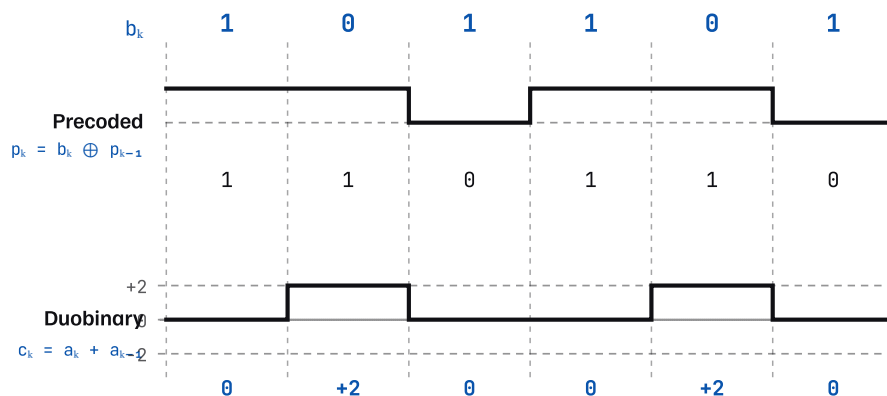
2+1

Explain how the same sequence will be decoded at the receiver from the received duobinary signal. C01

SOLUTION

Worked in full at the predicted Q6 above: precode by running XOR, map to ± 1 , add the previous sample to get the three-level signal 0, +2, 0, 0, +2, 0, and then decode with the memoryless rule $y = 0 \rightarrow 1, y = \pm 2 \rightarrow 0$, which returns 101101 exactly.

DRAWN FOR THE ANSWER



decode: $b_k = 0$ when $c_k = \pm 2$, and $b_k = 1$ when $c_k = 0$.

the precoder breaks the memory, so one bad symbol cannot poison the ones after it.

The precoder, and the three-level signal it produces. The precoder is what stops one bad symbol poisoning every symbol after it.

7 Probability of occurrence distribution of five symbols are given as follows: $P(A) = 0.40$, $P(B) = 0.20$, $P(C) = 0.15$, $P(D) = 0.15$, $P(E) = 0.10$. Construct the codes according to Shannon Fanno coding for these symbols. Calculate the Average code length, Entropy of the source, Efficiency and Redundancy. Further check whether the Kraft's inequality is being satisfied or not. C04

5

SOLUTION

Step 1 — sort and split. Already in descending order. Look for the split closest to half:

$\{A\} = 0.40$	vs $\{B, C, D, E\} = 0.60$	gap 0.20	
$\{A, B\} = 0.60$	vs $\{C, D, E\} = 0.40$	gap 0.20	(tie)

The tie is broken by taking the split nearer the top of the list, which gives the shorter code to the most likely symbol — and it turns out to give the better result, so take $\{A\} | \{B, C, D, E\}$.

```

A                → 0
{B, C, D, E} = 0.60 → 1

split: {B} = 0.20 vs {C, D, E} = 0.40    gap 0.20
       {B, C} = 0.35 vs {D, E} = 0.25    gap 0.10 ← closer

{B, C} → 10      {D, E} → 11
B = 100  C = 101  D = 110  E = 111

```

SYMBOL	P	CODE	L	$P \times L$
A	0.40	0	1	0.40
B	0.20	100	3	0.60
C	0.15	101	3	0.45
D	0.15	110	3	0.45
E	0.10	111	3	0.30

The four numbers:

$$L = 0.40 + 0.60 + 0.45 + 0.45 + 0.30 = 2.20 \text{ bits/symbol}$$

$$\begin{aligned} H(X) &= 0.40(1.3219) + 0.20(2.3219) + 0.15(2.7370) \\ &\quad + 0.15(2.7370) + 0.10(3.3219) \\ &= 0.5288 + 0.4644 + 0.4105 + 0.4105 + 0.3322 \\ &= 2.1464 \text{ bits/symbol} \end{aligned}$$

$$\eta = 2.1464 / 2.20 = 0.9757 \rightarrow 97.57 \%$$

$$\gamma = 1 - 0.9757 \rightarrow 2.43 \%$$

$$\text{Kraft} = 2^{-1} + 4 \times 2^{-3} = 0.5 + 0.5 = 1.00 \leq 1 \quad \text{SATISFIED}$$

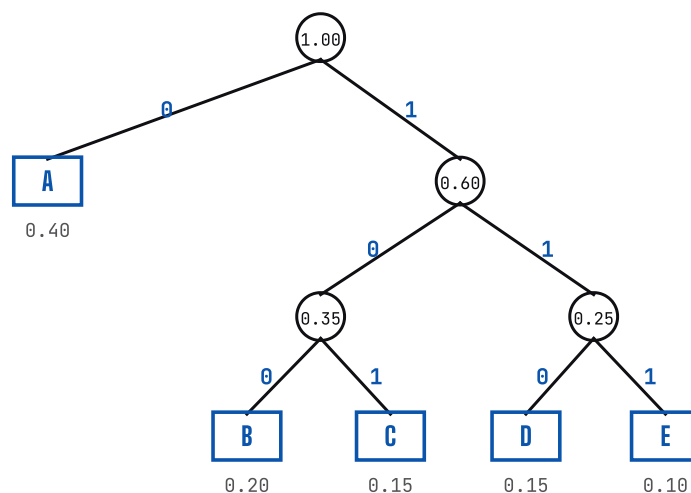
$$L = 2.20 \cdot H = 2.1464 \cdot \eta = 97.57\% \cdot \text{redundancy } 2.43\% \cdot \text{Kraft} = 1.00 \checkmark$$

THE TIE-BREAK IS WORTH KNOWING

Had you taken the other split — {A,B} against {C,D,E} — the lengths would be 2,2,2,3,3 and L would be **2.25**, an efficiency of 95.4%. Both are legitimate Shannon–Fano codes; one is simply better. **When the split is a tie, favour the one that gives the shortest codeword to the most probable symbol.**

DRAWN FOR THE ANSWER

SHANNON-FANO — SPLIT FROM THE TOP



Shannon-Fano splits the sorted list from the top; the codeword is the path down.

- 8 For the above question, instead of Shannon Fano coding, use Huffman coding and calculate the Average code length, Entropy of the source, Efficiency and Redundancy. Further check whether the Kraft's inequality is being satisfied or not. C04

5

SOLUTION

Build the tree bottom up — merge the two smallest each time and re-sort.

```

start   : 0.40  0.20  0.15  0.15  0.10
merge 1 : 0.15 + 0.10 = 0.25  → 0.40 0.25 0.20 0.15
merge 2 : 0.20 + 0.15 = 0.35  → 0.40 0.35 0.25
merge 3 : 0.35 + 0.25 = 0.60  → 0.60 0.40
merge 4 : 0.60 + 0.40 = 1.00  → done

```

Tracing back from the root gives depth 1 for A and depth 3 for all the others:

SYMBOL	P	CODE	L	P × L
A	0.40	0	1	0.40
B	0.20	100	3	0.60
C	0.15	101	3	0.45
D	0.15	110	3	0.45
E	0.10	111	3	0.30

```

L      = 2.20 bits/symbol
H(X)   = 2.1464 bits/symbol
eta    = 97.57 %
gamma  = 2.43 %
Kraft = 2-1 + 4 × 2-3 = 1.00 ≤ 1 SATISFIED

```

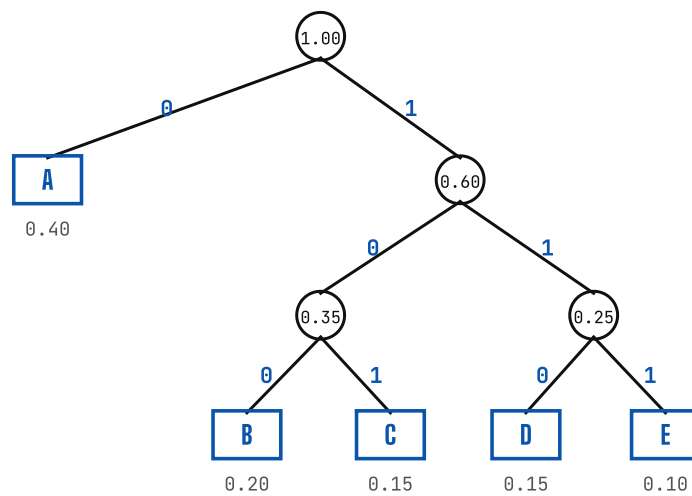
Identical to the Shannon–Fano result: $L = 2.20$, $\eta = 97.57\%$

The comparison the examiner wants. The two methods reach the same set of code lengths here — 1, 3, 3, 3, 3 — so they tie exactly. But they are not equivalent in general: **Huffman is provably optimal and Shannon–Fano is not.** Note that Shannon–Fano only matched it because the tie in its first split was broken the right way; break it the other way and it gives 2.25 while Huffman still gives 2.20.

That is the whole point of setting the same source twice: Huffman gets there *whatever* you do, Shannon–Fano needs luck.

DRAWN FOR THE ANSWER

HUFFMAN - MERGE FROM THE BOTTOM



Huffman merges the two smallest from the bottom. For this source it reaches the same lengths as Shannon-Fano, which is worth saying in the answer.

MID-SEM, SEPTEMBER 2024

EC305 · 30 MARKS

The last of the information-theory papers. Nine questions, and a lot of coding theory — Huffman, a systematic block code, and a cyclic code, all in the same paper.

S.V. National Institute of Technology, Surat

Department of Electronics Engineering

MID Semester Exam, September 2024

B. Tech. (EC) 5th Semester

DIGITAL COMMUNICATION (EC 305)

MARKS: 30

TIME: 1.5 HOURS

ANSWER ALL QUESTIONS IN SEQUENCE

- 1 A discrete memoryless source is described by the alphabet {A, B, C, D, E, F} with probabilities {1/32, 1/8, 1/4, 1/16, 1/32, 1/2} respectively. (a) Design a Huffman code and find the average length of the codeword. (b) Can you improve the Huffman code designed in part (a) by encoding the second-order extension of the source? Give reasons. C01

6

SOLUTION

(a) Sort and build.

```
sorted : 1/2   1/4   1/8   1/16   1/32   1/32
         0.5   0.25  0.125 0.0625 0.03125 0.03125   (sums to 1)

merge 1 : 1/32 + 1/32 = 1/16   → 1/2 1/4 1/8 1/16 1/16
merge 2 : 1/16 + 1/16 = 1/8    → 1/2 1/4 1/8 1/8
merge 3 : 1/8  + 1/8  = 1/4    → 1/2 1/4 1/4
merge 4 : 1/4  + 1/4  = 1/2    → 1/2 1/2
merge 5 : 1/2  + 1/2  = 1      → done
```

SYMBOL	P	CODE	L	P × L
F	1/2	0	1	0.5000
C	1/4	10	2	0.5000
B	1/8	110	3	0.3750
D	1/16	1110	4	0.2500
A	1/32	11110	5	0.1563
E	1/32	11111	5	0.1563

$$L = 0.5 + 0.5 + 0.375 + 0.25 + 0.15625 + 0.15625 = 1.9375 \text{ bits/symbol}$$

Now the entropy:

$$\begin{aligned} H(X) &= 1/2(1) + 1/4(2) + 1/8(3) + 1/16(4) + 1/32(5) + 1/32(5) \\ &= 0.5 + 0.5 + 0.375 + 0.25 + 0.15625 + 0.15625 \\ &= 1.9375 \text{ bits/symbol} \end{aligned}$$

$$L = H = 1.9375, \text{ so } \eta = 100\% \text{ exactly}$$

(b) No — and the reason is the whole point of the question.

Every probability here is an exact power of $1/2$, so each $-\log_2 P$ is a whole number: 1, 2, 3, 4, 5, 5. Huffman can therefore give every symbol a codeword of *precisely* its ideal length, and L hits the entropy exactly.

The source coding theorem says $L \geq H(X)$ for any code whatsoever. We are already at that floor, so no code can do better — and encoding the second-order extension, which gains at most $1/n$ bits per symbol for a source that is not already optimal, gains nothing here.

WHEN EXTENSION DOES HELP

Extension helps when the probabilities are *not* powers of one half, because then every ideal length $-\log_2 P$ is fractional and Huffman must round each one up to a whole number. Coding pairs of symbols lets that rounding waste be shared over two symbols instead of one. A source like this one — called **dyadic** — has no rounding waste to recover.

- 2 The parity check equations for a (6,3) systematic code are $b_0 = c_3 = m_0 + m_1 + m_2$, $b_1 = c_4 = m_0 + m_1$, $b_2 = c_5 = m_0 + m_2$. (a) Determine the generator matrix. (b) Determine the parity check matrix. (c) List all the code vectors. (d) What is the error correcting capability? (e) Decode the received codeword 101100. C01

5

SOLUTION

(a) **Generator matrix.** Systematic form is $G = [I_k \mid P]$, where column j of P is the contribution of each message bit to parity bit j .

$$\begin{aligned}
 b_0 &= m_0 + m_1 + m_2 && \rightarrow \text{P column 0} = (1,1,1) \\
 b_1 &= m_0 + m_1 && \rightarrow \text{P column 1} = (1,1,0) \\
 b_2 &= m_0 + m_2 && \rightarrow \text{P column 2} = (1,0,1)
 \end{aligned}$$

$$G = \begin{array}{ccc|ccc}
 & 1 & 0 & 0 & 1 & 1 & 1 \\
 & 0 & 1 & 0 & 1 & 1 & 0 \\
 & 0 & 0 & 1 & 1 & 0 & 1 \\
 \leftarrow I_3 \rightarrow & & & & \leftarrow P \rightarrow & &
 \end{array}$$

(b) Parity check matrix $H = [P^T \mid I_3]$:

$$H = \begin{array}{ccc|ccc}
 & 1 & 1 & 1 & 1 & 0 & 0 \\
 & 1 & 1 & 0 & 0 & 1 & 0 \\
 & 1 & 0 & 1 & 0 & 0 & 1
 \end{array}$$

(c) All eight code vectors, from $c = m \cdot G$:

M	B0 B1 B2	CODEWORD	WEIGHT
000	0 0 0	000 000	0
001	1 0 1	001 101	3
010	1 1 0	010 110	3
011	0 1 1	011 011	4
100	1 1 1	100 111	4
101	0 1 0	101 010	3
110	0 0 1	110 001	3
111	1 0 0	111 100	4

(d) **Error correcting capability.** For a linear code, $d_{\min}$ equals the smallest weight of any non-zero codeword. From the table that is **3**.

$$\begin{aligned}
 \text{detect} &\leq d_{\min} - 1 && = 2 \text{ errors} \\
 \text{correct} &\leq \text{floor}((d_{\min}-1)/2) && = 1 \text{ error}
 \end{aligned}$$

$d_{\min} = 3$, so it detects 2 errors and corrects 1

(e) Decode $r = 101100$. Compute the syndrome $s = H \cdot r^T$ (all arithmetic mod 2):

```
r = ( 1, 0, 1, 1, 0, 0 )
```

```
s0 = row0 . r = (1,1,1,1,0,0).(1,0,1,1,0,0) = 1+0+1+1+0+0 = 3 → 1
```

```
s1 = row1 . r = (1,1,0,0,1,0).(1,0,1,1,0,0) = 1+0+0+0+0+0 = 1 → 1
```

```
s2 = row2 . r = (1,0,1,0,0,1).(1,0,1,1,0,0) = 1+0+1+0+0+0 = 2 → 0
```

```
s = (1, 1, 0)
```

A non-zero syndrome means an error. Now match it against the columns of H:

```
col0 = (1,1,1)   col1 = (1,1,0) ← matches s
```

```
col2 = (1,0,1)   col3 = (1,0,0)
```

```
col4 = (0,1,0)   col5 = (0,0,1)
```

The syndrome equals **column 1**, so the error is in **bit position 1** (counting from 0), which is the second bit.

```
received : 1 0 1 1 0 0
```

```
flip bit 1: 1 1 1 1 0 0
```

```
corrected : 111100      -- which is in the table, for m = 111
```

Corrected codeword 111100, decoded message = 111

- 3 Taking $x^3 + x + 1$ as the generator polynomial for a (7,4) cyclic linear block code, determine the code vectors in systematic form for the message sequences (a) 1011 and (b) 1000. C03

5

SOLUTION

The method. For a systematic cyclic code with $n - k = 3$ parity bits:

1. multiply the message polynomial by $x^{(n-k)} = x^3$
2. divide by $g(x)$ and keep the remainder
3. codeword = $x^3 m(x) + \text{remainder}$

(a) $m = 1011$, so $m(x) = x^3 + x + 1$.

$$x^3 m(x) = x^6 + x^4 + x^3$$

divide by $g(x) = x^3 + x + 1$:

$$x^6 + x^4 + x^3 \quad / \quad x^3 + x + 1$$

$$\text{leading term: } x^6 / x^3 = x^3$$

$$x^3 (x^3 + x + 1) = x^6 + x^4 + x^3$$

$$\text{XOR: } (x^6 + x^4 + x^3) + (x^6 + x^4 + x^3) = 0$$

$$\text{remainder} = 0$$

$$c(x) = x^6 + x^4 + x^3 \rightarrow \text{codeword } 1011 \ 000$$

In bits: 1011000. The parity bits are all zero — which happens when $x^3 m(x)$ is already an exact multiple of $g(x)$, and it is a perfectly valid answer, not a mistake.

(b) $m = 1000$, so $m(x) = x^3$.

$$x^3 m(x) = x^6$$

$$x^6 / (x^3 + x + 1):$$

$$x^6 / x^3 = x^3 ; x^3 g = x^6 + x^4 + x^3$$

$$\text{XOR} \quad \rightarrow \quad x^4 + x^3$$

$$x^4 / x^3 = x ; x g = x^4 + x^2 + x$$

$$\text{XOR} \quad \rightarrow \quad x^3 + x^2 + x$$

$$x^3 / x^3 = 1 ; 1 g = x^3 + x + 1$$

$$\text{XOR} \quad \rightarrow \quad x^2 + 1$$

$$\text{remainder} = x^2 + 1 \rightarrow \text{parity bits } 101$$

$$c(x) = x^6 + x^2 + 1 \rightarrow \text{codeword } 1000 \ 101$$

Check both by dividing the whole codeword by 1011 — the remainder must be 000:

$$1011000 / 1011 : 1011 \text{ XOR } 1011 = 0000, \text{ then all zeros} \rightarrow \text{rem } 000 \text{ OK}$$

$$1000101 / 1011 : 1000 \text{ XOR } 1011 = 0011, \text{ bring } 1 \rightarrow 0111$$

$$\text{lead } 0, \text{ bring } 0 \rightarrow 1110$$

$$1110 \text{ XOR } 1011 = 0101, \text{ bring } 1 \rightarrow 1011$$

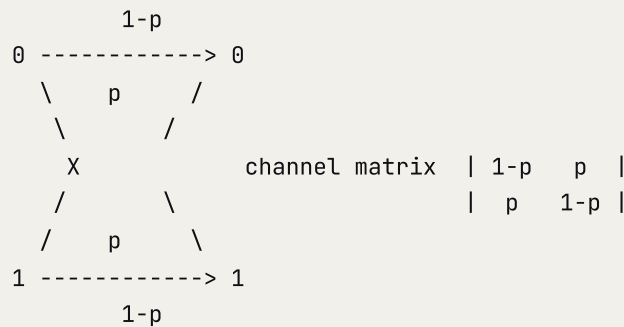
$$1011 \text{ XOR } 1011 = 0000 \rightarrow \text{rem } 000 \text{ OK}$$

- 5** Find the channel capacity of a binary symmetric channel. Sketch C against p , the transition probability, given that the input symbols are equiprobable. C03

3

SOLUTION

The BSC. Two inputs, two outputs, and one number describes it: p , the probability that a bit is flipped.



Capacity. With equiprobable inputs, $H(Y) = 1$ bit, and the conditional entropy $H(Y|X)$ is the entropy of the noise:

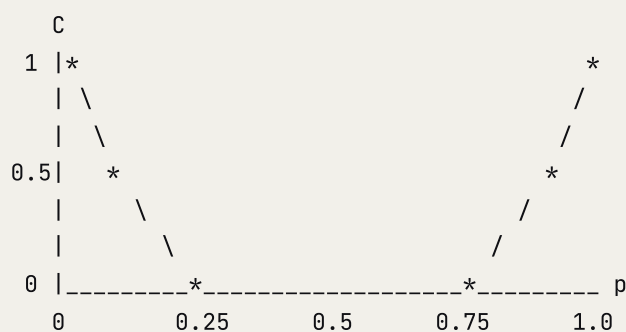
$$C = 1 + p \log_2 p + (1-p) \log_2 (1-p)$$

$$= 1 - H(p) \quad \text{where } H(p) \text{ is the binary entropy function}$$

Read the shape: **capacity = 1 minus what the noise steals**. The two log terms are always negative, and together they are exactly the entropy the channel injects.

Values to plot:

P	0	0.1	0.25	0.5	0.75	0.9	1.0
C	1.000	0.531	0.189	0	0.189	0.531	1.000

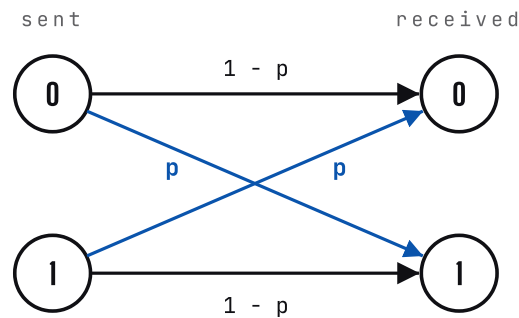


The three points to explain:

- $p = 0$ — a perfect channel, $C = 1$ bit per symbol.
- $p = 0.5$ — the *worst possible* channel, $C = 0$. The output is a pure coin toss with no relation to the input, so you learn nothing.
- $p = 1$ — every bit is flipped, and C is back to 1. This sounds terrible but is actually perfect: just invert everything at the receiver. That is why p is normally

restricted to $0 \leq p \leq \frac{1}{2}$ — anything beyond is a relabelling of something below.

DRAWN FOR THE ANSWER



$C = 1 - H(p)$ — capacity is 1 at $p = 0$ and at $p = 1$, and 0 at $p = 0.5$

The binary symmetric channel. Capacity is 1 at $p = 0$ and at $p = 1$, and falls to zero at $p = 0.5$, where the output tells you nothing.

- 6 Using 4×10^5 pixels per frame, with each pixel taking any one of 10 different brightness levels with equal probability, a black and white TV transmitter transmits 25 frames per second. If the signal-to-noise ratio is 30 dB, determine the minimum bandwidth required to transmit the video signal. Assume pixels to be statistically independent. C01

4

SOLUTION

Step 1 — information per pixel. Ten equally likely levels:

$$H = \log_2(10) = 3.3219 \text{ bits per pixel}$$

Step 2 — information rate. Pixels are independent, so rates simply multiply:

$$\text{pixels per second} = 4 \times 10^5 \times 25 = 10^7$$

$$R = 10^7 \times 3.3219 = 3.322 \times 10^7 \text{ bits/s} = 33.22 \text{ Mb/s}$$

Step 3 — convert the SNR out of dB. This is the step most often skipped:

$$30 = 10 \log_{10}(S/N) \rightarrow S/N = 10^3 = 1000$$

Step 4 — Shannon–Hartley, rearranged for B. For error-free transmission we need $C \geq R$:

$$C = B \log_2(1 + S/N) \geq R$$

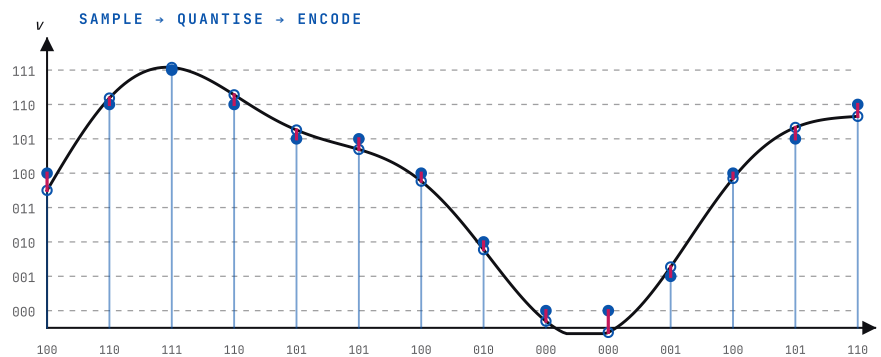
$$\begin{aligned} B &\geq R / \log_2(1001) \\ &= 3.322 \times 10^7 / 9.967 \\ &= 3.333 \times 10^6 \text{ Hz} \end{aligned}$$

$$B(\text{min}) = 3.33 \text{ MHz}$$

SANITY CHECK

A real analogue TV channel is about 5–6 MHz wide, so 3.33 MHz for this picture is the right order of magnitude. If your answer had come out in kHz or in hundreds of MHz, something went wrong — almost always the dB conversion.

DRAWN FOR THE ANSWER



the hollow ring is the true sample; the filled dot is the level it was rounded to.
the short pink stub between them is the quantisation error, and it never exceeds half a step.
 $SNR_s = 1.76 + 6.02n \text{ dB}$ — one more bit is six more decibels, always.

Sample, quantise, encode — and the rounding error that the third step can never get back.

- 8 A random binary data sequence 1 1 0 1 0 0 1 0 1 is transmitted. Sketch the line-coded version for (a) NRZ-L, (b) NRZ-I, (c) Manchester, (d) Differential Manchester. C02

4

SOLUTION

```
bit : 1 1 0 1 0 0 1 0 1
```

NRZ-L (Level: the LEVEL itself carries the bit)

```
+A |___|___|   |___|   |___|   |___|
-A |         |___|   |___|___|   |___|
```

NRZ-I (Inversion: a TRANSITION at the start means a 1)

start at -A, invert whenever the bit is 1

```
bit 1 1 0 1 0 0 1 0 1
```

```
level +A -A -A +A +A +A -A -A +A
```

```
| | | |   | | |
```

```
inv inv - inv - - inv - inv
```

MANCHESTER (transition in the MIDDLE of every bit)

1 = low-to-high, 0 = high-to-low

```
_| _| |_ _| |_ |_ _| |_ _|
```

DIFF. MANCHESTER (always a mid-bit transition for the clock;
a 0 ALSO puts a transition at the START of the bit,
a 1 does not)

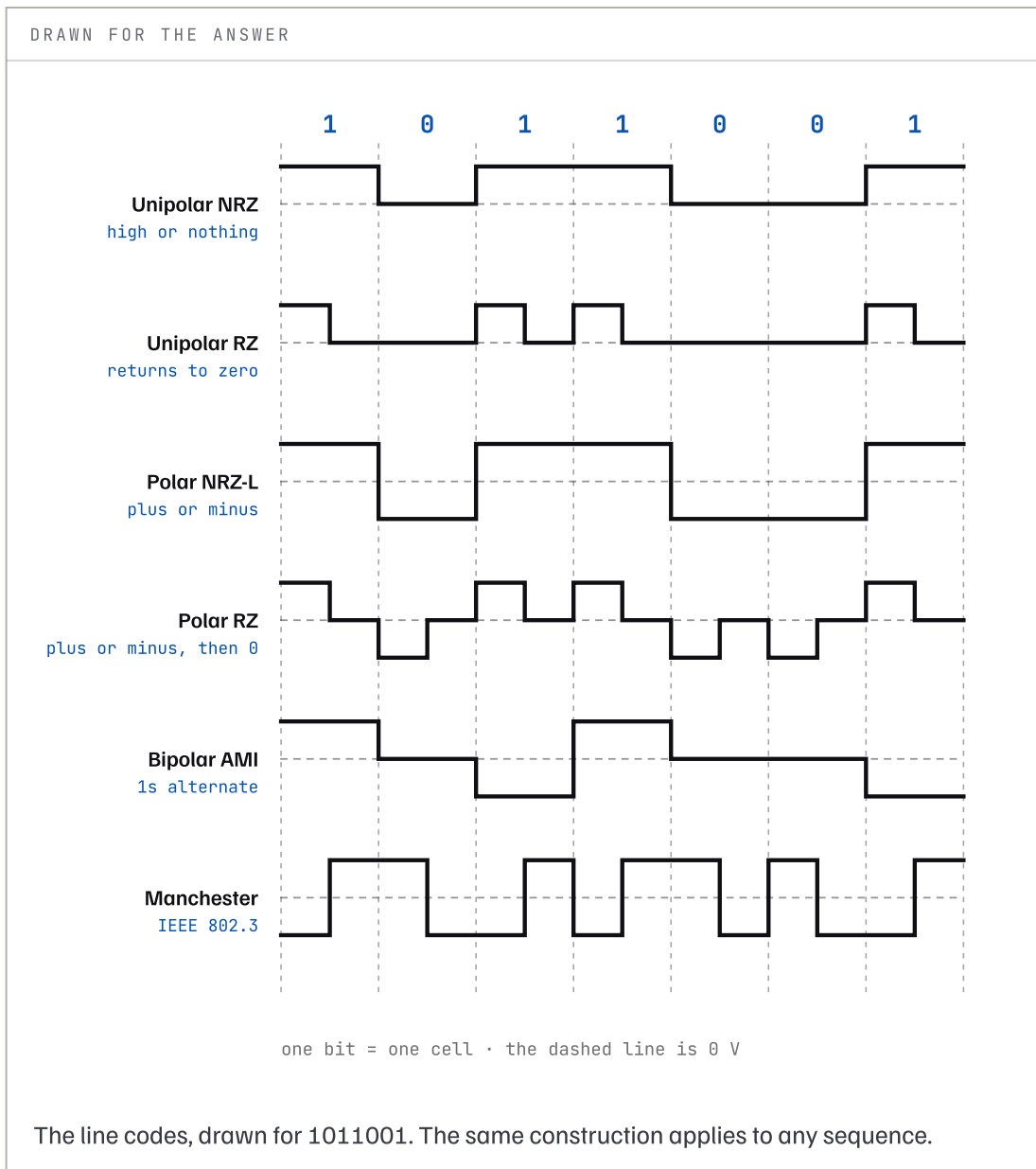
```
bit 1 1 0 1 0 0 1 0 1
```

```
start - - inv - inv inv - inv -
```

```
mid inv inv inv inv inv inv inv inv inv
```

CODE	WHAT CARRIES THE BIT	SELF-CLOCKING?	POLARITY-PROOF?
NRZ-L	the voltage level	No	No — swap the wires and every bit inverts
NRZ-I	the presence of a transition	Only on 1s	Yes — it reads changes, not levels
Manchester	the direction of the mid-bit transition	Yes, always	No
Diff. Manchester	presence or absence of a transition at the bit boundary	Yes, always	Yes

The trade-off to state in one line: the two Manchester codes are self-clocking whatever the data, and pay for it with **twice the bandwidth** of NRZ, because there are up to two transitions per bit instead of one.



- 9 Answer any two: (i) line coding suppresses the DC component. (ii) The ARQ system with the highest throughput efficiency is and the one with the lowest is (iii) For a linear (7,4) block code, the ratio of parity bits to message bits is and the code rate is (iv) The mutual information between two discrete random variables X and Y is given by C04

2

SOLUTION

- (i) **Bipolar (AMI)** — and Manchester also qualifies. AMI works because successive 1s alternate in polarity, so the long-run average is zero. Manchester works because every bit contains one half at each level.

(ii) Highest: Selective Repeat · Lowest: Stop-and-Wait

Stop-and-Wait puts one frame in flight per round trip, so on a long link it can sit at a few per cent. Go-Back-N fills the pipe but wastes it on retransmissions after a loss. Selective Repeat resends only the lost frame, so it is the most efficient of the three — at the cost of receiver buffers and windows capped at 2^{m-1} .

(iii) parity : message = 3 : 4 · code rate $R = k/n = 4/7 \approx 0.571$

$n = 7$ total bits, $k = 4$ message bits, so $n - k = 3$ parity bits. About 57% of what goes on the wire is data.

(iv)

$$\begin{aligned} I(X;Y) &= H(X) - H(X|Y) \\ &= H(Y) - H(Y|X) \\ &= H(X) + H(Y) - H(X,Y) \end{aligned}$$

In words: how much the uncertainty about X is *reduced* by observing Y . It is symmetric, it is never negative, and it is zero exactly when X and Y are independent. Maximising it over the input distribution gives the channel capacity.

MID-SEM, OCTOBER 2023

EC303 · 30 MARKS · ANY 5 OF 6

Six questions of six marks each, of which you attempt any five. Almost entirely information theory.

Sardar Vallabhbhai National Institute of Technology Surat

Electronics Engineering Department

B.Tech III (Sem 5) Mid Sem Exam — October 2023

DIGITAL COMMUNICATION (EC303)

MARKS: 30

TIME: 1.5 HOURS

ATTEMPT ANY 5 QUESTIONS · 6 MARKS EACH

- Q-1** A 7-bit PCM system employing uniform quantization has an overall signalling rate of 56 kbps. What is the theoretical maximum frequency f_m (bandwidth) that this system can handle? Based on a channel SNR of 35 dB, what is the channel capacity for error-free transmission? If this SNR is dropped by 50%, what will be the percentage drop in channel capacity? [Hint: signalling rate r = number of bits $n \times$ sampling rate f_s] 6

SOLUTION

Step 1 — the sampling rate, from the hint.

$$r = n \times f_s$$

$$56,000 = 7 \times f_s \quad \rightarrow \quad f_s = 8,000 \text{ samples/s}$$

Step 2 — maximum signal frequency, from Nyquist.

$$f_s \geq 2 f_m \quad \rightarrow \quad f_m = f_s / 2 = 4,000 \text{ Hz} = 4 \text{ kHz}$$

$f_m = 4 \text{ kHz}$ — which is exactly telephone quality, and not a coincidence

Step 3 — channel capacity at 35 dB. Convert the dB first:

$$35 = 10 \log_{10}(S/N) \quad \rightarrow \quad S/N = 10^{3.5} = 3162.3$$

$$\begin{aligned} C &= B \log_2(1 + S/N) \quad \text{with } B = f_m = 4000 \text{ Hz} \\ &= 4000 \times \log_2(3163.3) \\ &= 4000 \times 11.627 \\ &= 46,509 \text{ bits/s} \end{aligned}$$

$$C \approx 46.5 \text{ kb/s}$$

Step 4 — drop the SNR by 50%. Read the words carefully: the SNR *ratio* is halved, not the dB figure.

$$\text{new } S/N = 3162.3 / 2 = 1581.1$$

$$\begin{aligned} C_2 &= 4000 \times \log_2(1581.1) \\ &= 4000 \times 10.628 \\ &= 42,511 \text{ bits/s} \end{aligned}$$

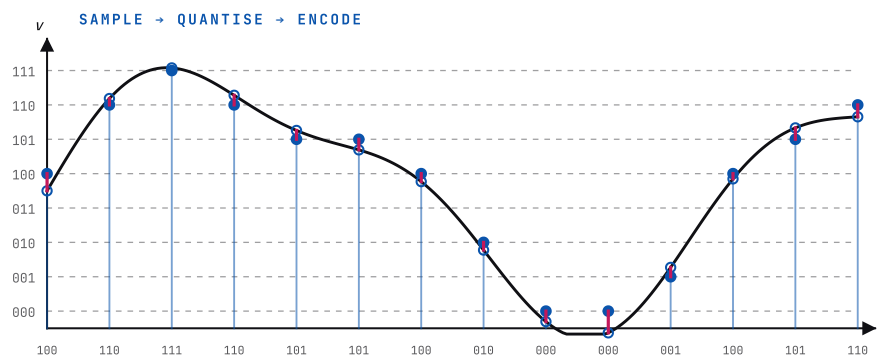
$$\begin{aligned} \text{percentage drop} &= (46,509 - 42,511) / 46,509 \times 100 \\ &= 8.60 \% \end{aligned}$$

Capacity falls by only 8.6%

THE COMMENT THAT FINISHES THE ANSWER

Halving the signal power — a 3 dB loss — costs less than 9% of the capacity. That is because power sits *inside* the logarithm. Halving the **bandwidth** instead would have cost exactly 50%, because bandwidth sits outside it. **Bandwidth is far more valuable than power**, and this question is set to make you notice.

DRAWN FOR THE ANSWER



the hollow ring is the true sample; the filled dot is the level it was rounded to.
the short pink stub between them is the quantisation error, and it never exceeds half a step.
 $SNR_s = 1.76 + 6.02n \text{ dB}$ — one more bit is six more decibels, always.

The three steps of PCM, and where the quantisation error comes from.

- Q-2** (a) What is the importance of the Kraft inequality condition? (b) Draw the model of a Binary Symmetric Channel. (c) An instantaneous adaptive delta modulator adapts its step size exponentially: if the predictor cannot track the input, the step size is doubled, so the steps are $S_n = 2^n$ for $n = 1, 2, \dots, 10$. Find the entropy of

the step generator if the probability of a step S_n is $p(S_n) = 1/S_n$ for $n = 1 \dots 9$, and $p(S_{10}) = p(S_9)$. C01

SOLUTION

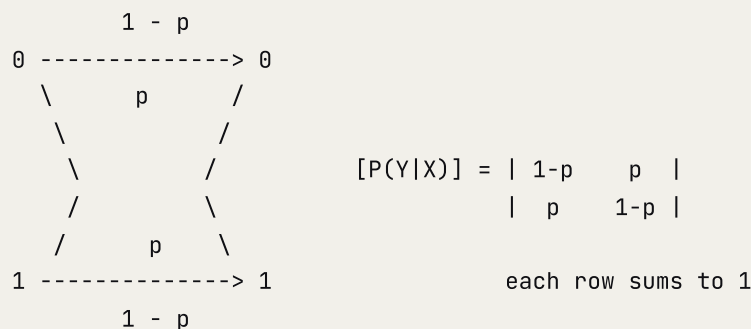
(a) Kraft inequality.

$$K = \sum 2^{-l_i} \leq 1$$

It is a **necessary and sufficient** condition for an instantaneous (prefix-free) binary code with those codeword lengths to exist. Think of it as a budget: you start with 1, a codeword of length l costs 2^{-l} , so short codewords are expensive. Overspend and the codewords would be forced to collide.

Its importance is that it lets you **rule a set of lengths in or out before building anything**. But note what it does not do: it tells you such a code exists without showing you the codewords, and it does not mean any arbitrary code with those lengths is uniquely decodable. For the construction you still need Shannon–Fano or Huffman.

(b) BSC model.



“Symmetric” because a 0 turning into a 1 is exactly as likely as a 1 turning into a 0. One number, p , describes the whole channel.

(c) Entropy of the step generator.

$$S_n = 2^n, \text{ so } p(S_n) = 1/2^n \text{ for } n = 1 \dots 9$$

$$p(S_{10}) = p(S_9) = 1/512$$

Check the probabilities sum to 1 — always do this first:

$$\begin{aligned}
 \text{sum}_{\{n=1..9\}} 1/2^n &= 1 - 1/512 = 511/512 \\
 \text{plus } p(S_{10}) &= 1/512 \\
 &\text{-----} \\
 &512/512 = 1 \quad \text{OK}
 \end{aligned}$$

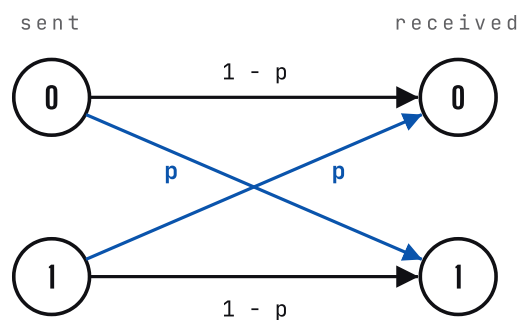
Now the entropy. For $n = 1..9$, $\log_2(1/p) = n$ exactly, which makes the sum easy:

$$\begin{aligned}
 H &= \text{sum}_{\{n=1..9\}} (1/2^n)(n) + (1/512)(9) \\
 \text{the standard result: } \text{sum}_{\{n=1..N\}} n/2^n &= 2 - (N+2)/2^N \\
 \text{with } N = 9 : &= 2 - 11/512 = 2 - 0.021484 = 1.978516 \\
 H &= 1.978516 + 9/512 \\
 &= 1.978516 + 0.017578 \\
 &= 1.996094 \text{ bits}
 \end{aligned}$$

$H \approx 1.996$ bits per step, essentially 2 bits

Worth noticing: the maximum for ten symbols would be $\log_2 10 = 3.32$ bits. This source only reaches 2, because the probabilities are heavily lopsided — half the time it is the smallest step.

DRAWN FOR THE ANSWER



$C = 1 - H(p)$ — capacity is 1 at $p = 0$ and at $p = 1$, and 0 at $p = 0.5$

The binary symmetric channel model the question asks you to draw.

- Q-3** A DMS X has six symbols $x_1 \dots x_6$ with $p(x_1) = 0.3$, $p(x_2) = 0.19$, $p(x_3) = 0.16$, $p(x_4) = 0.15$, $p(x_5) = 0.1$, $p(x_6) = 0.1$. (a) Construct a Shannon–Fano code for X and calculate the efficiency of the code. (b) Repeat for Huffman coding and compare the results. C03

SOLUTION

Entropy first, since both parts need it:

$$\begin{aligned}
 H(X) &= 0.30(1.7370) + 0.19(2.3959) + 0.16(2.6439) \\
 &\quad + 0.15(2.7370) + 0.10(3.3219) + 0.10(3.3219) \\
 &= 0.5211 + 0.4552 + 0.4230 + 0.4105 + 0.3322 + 0.3322 \\
 &= 2.4743 \text{ bits/symbol}
 \end{aligned}$$

(a) Shannon–Fano. Split into two groups of as nearly equal probability as possible:

$$\begin{aligned}
 \{x_1, x_2\} &= 0.49 \quad \text{vs} \quad \{x_3, x_4, x_5, x_6\} = 0.51 \quad \text{gap } 0.02 \text{ (excellent)} \\
 &\rightarrow 0 \quad \quad \quad \rightarrow 1 \\
 \{x_1\} &= 0.30 \text{ vs } \{x_2\} = 0.19 \quad \rightarrow x_1 = 00, \quad x_2 = 01 \\
 \{x_3, x_4\} &= 0.31 \text{ vs } \{x_5, x_6\} = 0.20 \quad \text{gap } 0.11 \\
 &\rightarrow 10 \quad \quad \quad \rightarrow 11 \\
 x_3 &= 100, \quad x_4 = 101 \quad \quad \quad x_5 = 110, \quad x_6 = 111
 \end{aligned}$$

lengths : 2, 2, 3, 3, 3, 3

$$\begin{aligned}
 L &= 0.30(2) + 0.19(2) + 0.16(3) + 0.15(3) + 0.10(3) + 0.10(3) \\
 &= 0.60 + 0.38 + 0.48 + 0.45 + 0.30 + 0.30 \\
 &= 2.51 \text{ bits/symbol}
 \end{aligned}$$

$$\eta = 2.4743 / 2.51 = 0.9858 \rightarrow 98.58 \%$$

(b) Huffman. Merge the two smallest each time:

$$\begin{aligned}
 \text{start} &: 0.30 \ 0.19 \ 0.16 \ 0.15 \ 0.10 \ 0.10 \\
 \text{merge 1} &: 0.10+0.10 = 0.20 \rightarrow 0.30 \ 0.20 \ 0.19 \ 0.16 \ 0.15 \\
 \text{merge 2} &: 0.16+0.15 = 0.31 \rightarrow 0.31 \ 0.30 \ 0.20 \ 0.19 \\
 \text{merge 3} &: 0.20+0.19 = 0.39 \rightarrow 0.39 \ 0.31 \ 0.30 \\
 \text{merge 4} &: 0.31+0.30 = 0.61 \rightarrow 0.61 \ 0.39 \\
 \text{merge 5} &: 0.61+0.39 = 1.00 \rightarrow \text{done}
 \end{aligned}$$

Tracing back gives depths 2, 2, 3, 3, 3, 3 — the same length profile as Shannon–Fano, though the codewords themselves differ.

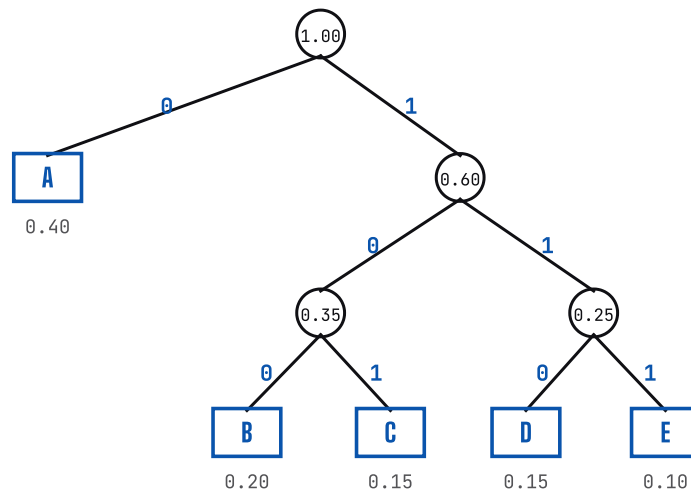
$$\begin{aligned}
 L &= 2.51 \text{ bits/symbol} \\
 \eta &= 98.58 \%
 \end{aligned}$$

Both give $L = 2.51$ and $\eta = 98.58\%$ — a dead heat

The comparison. They tie here because the first Shannon–Fano split was almost perfect (0.49 against 0.51), which is exactly the condition under which the top-down method matches the optimal bottom-up one. **Huffman is optimal by construction; Shannon–Fano only ever matches it by luck.** On a source where no split lands near half, Shannon–Fano falls behind — which is why only Huffman ended up inside JPEG, MP3 and ZIP.

DRAWN FOR THE ANSWER

HUFFMAN - MERGE FROM THE BOTTOM



How a Huffman tree is built — merge the two smallest, put the new node back, repeat.

MID-SEM, OCTOBER 2022

EC303 · 30 MARKS · ANY 5 OF 6

The oldest paper we have, and the closest in style to 2023. Useful mainly for the two numericals that do not appear anywhere else — the voice-over-a-narrow-channel question and the four-level source.

Sardar Vallabhbhai National Institute of Technology Surat

Electronics Engineering Department

B.Tech III (Sem 5) Mid Sem Exam — October 2022

DIGITAL COMMUNICATION (EC303)

MARKS: 30

TIME: 1.5 HOURS

ATTEMPT ANY 5 QUESTIONS · 6 MARKS EACH

- Q-3** Digital voice of 32 kbps is to be transmitted through a bandwidth of 3.1 kHz. The transmitted power is such that the received signal-to-noise ratio is 1200. Determine whether the data can be transmitted error free through this channel. If not, determine the minimum SNR required for error-free transmission, and the spectrum efficiency in this case. What is E_b/N_0 for the received data? 6

SOLUTION

Step 1 — what the channel can actually carry.

$$\begin{aligned}C &= B \log_2(1 + S/N) \\&= 3100 \times \log_2(1 + 1200) \\&= 3100 \times \log_2(1201) \\&= 3100 \times 10.230 \\&= 31,713 \text{ bits/s}\end{aligned}$$

Step 2 — compare with what is needed.

$$\begin{aligned}\text{required } R &= 32,000 \text{ bits/s} \\ \text{available } C &= 31,713 \text{ bits/s}\end{aligned}$$

$$R > C$$

No — error-free transmission is impossible. It falls short by only 287 bit/s, but short is short.

Shannon's theorem is a wall, not a guideline: when $R > C$, *no* coding scheme whatsoever can give an arbitrarily low error rate.

Step 3 — the minimum SNR that would work. Set $C = 32,000$ and solve backwards:

$$\begin{aligned} 32,000 &= 3100 \log_2(1 + S/N) \\ \log_2(1 + S/N) &= 32,000 / 3100 = 10.3226 \\ 1 + S/N &= 2^{10.3226} = 1280.6 \\ S/N &= 1279.6 \quad (31.07 \text{ dB}) \end{aligned}$$

Minimum $S/N \approx 1280$, i.e. 31.07 dB — only about 6.6% more than the 1200 available

Step 4 — spectrum efficiency.

$$\eta_{\text{spectral}} = R / B = 32,000 / 3100 = 10.32 \text{ bits/s per Hz}$$

That is a demanding figure — it needs roughly 1024-QAM-class signalling — which is exactly why the required SNR is so high.

Step 5 — E_b/N_0 . Use the identity that links it to S/N :

$$\begin{aligned} S/N &= (E_b/N_0) \times (R/B) \\ \text{so } E_b/N_0 &= (S/N) / (R/B) \\ &= 1279.6 / 10.32 \\ &= 123.96 \quad (20.93 \text{ dB}) \end{aligned}$$

$E_b/N_0 \approx 124$, about 20.9 dB

WHERE THAT IDENTITY COMES FROM

$S = E_b \cdot R$ (energy per bit times bits per second is power) and $N = N_0 \cdot B$ (noise density times bandwidth). Dividing one by the other gives $S/N = (E_b/N_0)(R/B)$ immediately. It is the single most useful line in this whole topic, and it is worth deriving in the margin rather than memorising.

For reference, the absolute Shannon limit is $E_b/N_0 = -1.59 \text{ dB}$, but that is only approached as the spectral efficiency tends to zero. At 10.32 bits/s/Hz you are nowhere near it, and 20.9 dB is the price of packing that many bits into that little bandwidth.

- Q-4** A DMS X has five symbols $x_1 \dots x_5$ with $p(x_1) = 0.4$, $p(x_2) = 0.19$, $p(x_3) = 0.16$, $p(x_4) = 0.15$, $p(x_5) = 0.1$. (i) Construct a Shannon–Fano code for X and calculate the efficiency of the code. (ii) Repeat for Huffman coding and compare the results. C03

SOLUTION**Entropy:**

$$\begin{aligned}
 H(X) &= 0.40(1.3219) + 0.19(2.3959) + 0.16(2.6439) \\
 &\quad + 0.15(2.7370) + 0.10(3.3219) \\
 &= 0.5288 + 0.4552 + 0.4230 + 0.4105 + 0.3322 \\
 &= 2.1498 \text{ bits/symbol}
 \end{aligned}$$

(i) Shannon–Fano.

```

{x1} = 0.40 vs rest = 0.60          gap 0.20
{x1,x2} = 0.59 vs {x3,x4,x5} = 0.41 gap 0.18 ← closer, take this

x1, x2 → 0          x3, x4, x5 → 1
x1 = 00, x2 = 01

{x3} = 0.16 vs {x4,x5} = 0.25      gap 0.09 ← closer
x3 = 10          {x4,x5} → 11
x4 = 110, x5 = 111

```

```

lengths : 2, 2, 2, 3, 3

```

$$\begin{aligned}
 L &= 0.40(2) + 0.19(2) + 0.16(2) + 0.15(3) + 0.10(3) \\
 &= 0.80 + 0.38 + 0.32 + 0.45 + 0.30 \\
 &= 2.25 \text{ bits/symbol}
 \end{aligned}$$

$$\text{eta} = 2.1498 / 2.25 = 0.9554 \rightarrow 95.54 \%$$

(ii) Huffman.

```

start   : 0.40 0.19 0.16 0.15 0.10
merge 1 : 0.15 + 0.10 = 0.25 → 0.40 0.25 0.19 0.16
merge 2 : 0.19 + 0.16 = 0.35 → 0.40 0.35 0.25
merge 3 : 0.35 + 0.25 = 0.60 → 0.60 0.40
merge 4 : 0.60 + 0.40 = 1.00 → done

```

x_1 is merged only at the last step, so it gets a **1-bit** codeword; everything else sits at depth 3.

```
lengths : 1, 3, 3, 3, 3

L = 0.40(1)+0.19(3)+0.16(3)+0.15(3)+0.10(3)
  = 0.40 + 0.57 + 0.48 + 0.45 + 0.30
  = 2.20 bits/symbol

eta = 2.1498 / 2.20 = 0.9772 → 97.72 %
```

The comparison — and this time they do not tie.

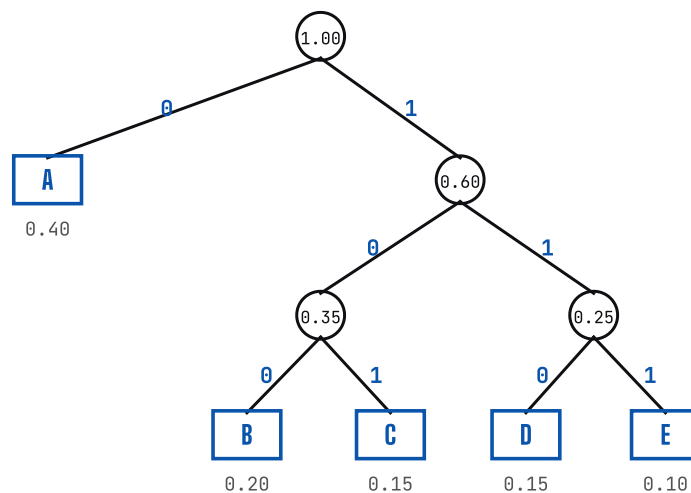
	SHANNON-FANO	HUFFMAN
Lengths	2, 2, 2, 3, 3	1, 3, 3, 3, 3
L	2.25	2.20
Efficiency	95.54%	97.72%

Huffman wins by 0.05 bits per symbol – 2.2 percentage points of efficiency

Why. Shannon–Fano works top-down and is forced to make its first cut somewhere near the middle, so the 0.40 symbol gets stuck in a two-symbol group and ends up with 2 bits. Huffman works bottom-up: the most probable symbol is merged *last*, which automatically leaves it nearest the root with the shortest codeword. That is the structural reason **Huffman is optimal and Shannon–Fano is not** — and this source shows it, where the 2023 and 2025 sources happened to hide it.

DRAWN FOR THE ANSWER

HUFFMAN - MERGE FROM THE BOTTOM



The Huffman construction.

- Q-6** An analog signal is bandlimited to 6 kHz and sampled at the Nyquist rate. The samples are quantized into 4 levels, each level representing one symbol. The probabilities of these 4 levels are $p(x_1) = p(x_4) = 1/8$ and $p(x_2) = p(x_3) = 3/8$. Obtain the information rate R of this source. If this source were an equiprobable DMS, what would the information rate be? If the AWGN channel SNR is 42 dB, find the theoretical limit of the bit rate C so that $H(X) \leq R \leq C$. C01

SOLUTION

Step 1 — symbol rate. Sampling at the Nyquist rate:

$f_s = 2 \times 6 \text{ kHz} = 12,000 \text{ samples/s}$
 one symbol per sample, so $r = 12,000 \text{ symbols/s}$

Step 2 — entropy of the source.

$$H(X) = 2 \times (1/8) \log_2(8) + 2 \times (3/8) \log_2(8/3)$$

$$\log_2(8) = 3$$

$$\log_2(8/3) = \log_2(2.6667) = 1.4150$$

$$\begin{aligned} H &= 2(0.125)(3) + 2(0.375)(1.4150) \\ &= 0.75 + 1.0613 \\ &= 1.8113 \text{ bits/symbol} \end{aligned}$$

Step 3 — information rate.

$$R = r \times H(X) = 12,000 \times 1.8113 = 21,735 \text{ bits/s}$$

$$R \approx 21.74 \text{ kb/s}$$

Step 4 — if the source were equiprobable. Four equally likely symbols give the maximum possible entropy:

$$\begin{aligned} H &= \log_2(4) = 2 \text{ bits/symbol} \\ R &= 12,000 \times 2 = 24,000 \text{ bits/s} \end{aligned}$$

$R = 24 \text{ kb/s}$ — the maximum, and 2.27 kb/s more than the real source

That difference is the source's **redundancy**: because the middle two levels occur three times as often as the outer two, the sequence is partly predictable, and a source coder could exploit it.

Step 5 — the channel limit.

$$42 = 10 \log_{10}(S/N) \rightarrow S/N = 10^{4.2} = 15,849$$

$$\begin{aligned} C &= B \log_2(1 + S/N) \\ &= 6000 \times \log_2(15,850) \\ &= 6000 \times 13.952 \\ &= 83,713 \text{ bits/s} \end{aligned}$$

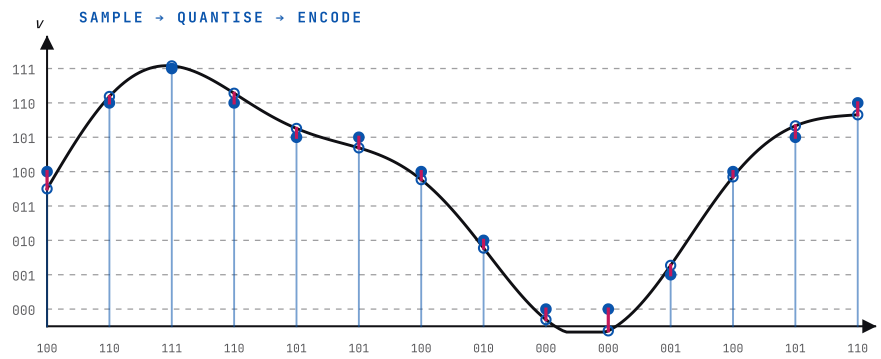
$$C \approx 83.7 \text{ kb/s}$$

Check the condition:

$$\begin{array}{rclcl} H(X) \times r & \leq & R & \leq & C \\ 21,735 & \leq & 21,735 & \leq & 83,713 \end{array} \quad \text{SATISFIED, with plenty to spare}$$

The channel can carry nearly four times what this source produces, so error-free transmission is comfortably possible — and there is capacity left over for the channel coding that makes it so.

DRAWN FOR THE ANSWER



the hollow ring is the true sample; the filled dot is the level it was rounded to.
 the short pink stub between them is the quantisation error, and it never exceeds half a step.
 $SNR_s = 1.76 + 6.02n$ dB – one more bit is six more decibels, always.

Sampling and quantising, which is where the bit rate in this question comes from.

EC301 · DIGITAL COMMUNICATION · PAPERS WORKED BY HAND AND CHECKED NUMERICALLY
 THE PREDICTED PAPER IS OUR OWN READING OF THE PATTERN – IT IS NOT AN OFFICIAL ANYTHING, AND
 IT IS NOT A LEAK
 BACK TO TU CHAHYE · THE ORIGINAL SCANS · ALL THE PDFS